Blink: CPU-Free LLM Inference by Delegating the Serving Stack to GPU and SmartNIC

Mohammad Siavashi
KTH Royal Institute of Technology
Stockholm, Sweden

Mariano Scazzariello
RISE
Stockholm, Sweden

Gerald Q. Maguire Jr.
KTH Royal Institute of Technology
Stockholm, Sweden

Dejan Kostić
KTH Royal Institute of Technology
Stockholm, Sweden

Marco Chiesa
KTH Royal Institute of Technology
Stockholm, Sweden

arXiv:2604.07609v1 [cs.DC] 8 Apr 2026

Abstract

Large Language Model (LLM) inference is rapidly becoming a core datacenter service, yet current serving stacks keep the host CPU on the critical path for orchestration and token-level control. This makes LLM performance sensitive to CPU interference, undermining application colocation and forcing operators to reserve CPU headroom, leaving substantial capacity unutilized.

We introduce Blink, an end-to-end serving architecture that *removes* the host CPU from the steady-state inference path by redistributing responsibilities across a SmartNIC and a GPU. Blink offloads request handling to the SmartNIC, which delivers inputs directly into GPU memory via RDMA, and replaces host-driven scheduling with a *persistent* GPU kernel that performs batching, scheduling, and KV-cache management *without* CPU involvement. Evaluated against TensorRT-LLM, vLLM, and SGLang, Blink outperforms all baselines even in isolation, reducing pre-saturation P99 time-to-first-token (TTFT) by up to 8.47× and P99 time-per-output-token (TPOT) by up to 3.40×, improving decode throughput by up to 2.1×, and reducing energy per token by up to 48.6 %. Under CPU interference, Blink maintains stable performance, while existing systems degrade by up to two orders of magnitude.

1 Introduction

Large Language Models (LLMs) are a foundational technology for modern Artificial Intelligence (AI) systems, powering search, translation, summarization, assistants, coding, and enterprise automation. As deployments scale to large user bases [8], including latency-sensitive applications [51, 57], inference cost and complexity are becoming primary concerns for datacenter operators [38, 44]. Although inference pipelines rely heavily on GPUs for tensor computation, standard architectures still center the server CPU for request handling, orchestration, and data movement [20, 43, 54].

LLM serving systems are increasingly CPU-dependent. As quality-of-service demands become stricter, the control and data planes running on the server are growing increasingly complex. For example, after Orca [54] introduced iteration-level scheduling, many systems adopted this approach, which requires handing control back to the CPU of the server after every token (or every few tokens). Moreover, recent control-plane optimizations such as chunked prefill [2], which splits long prompts into smaller chunks and interleaves them with decode tokens to reduce latency stalls, further complicate CPU-side logic. Although these techniques improve LLM serving performance, they also add significant CPU-side overheads, making inference susceptible to interference from colocated workloads [34].

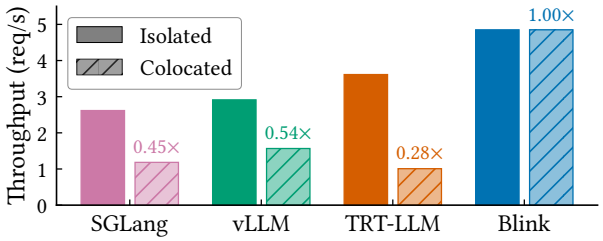


Figure 1. Achieved throughput of 4 LLM serving systems on Qwen-3 30B-A3B (MoE) at 4 req/s offered load, under isolated and colocated execution. Blink is unaffected by colocation while baselines degrade. Annotations show the colocated-to-isolated throughput ratio.

The impact of CPU interference. The dominant LLM deployment model dedicates an entire machine to a single workload, simplifying Service Level Objective (SLO) management and predictability but often leaves CPU resources stranded [12, 38, 44]. Reclaiming underused CPU resources on machines running GPU-intensive LLM workloads, *without* degrading performance, would allow datacenters and cloud providers to generate additional revenue from the hundreds of thousands of machines currently dedicated to LLM inference. We quantify this fragility in Figure 1: under colocation with CPU-intensive workloads, three production serving systems (SGLang [56], vLLM [20], TRT-LLM [26]) retain only 28–54% of their isolated throughput, highlighting the impact of CPU interference on inference performance.

Existing SmartNIC offloads primarily target the data path. LLM inference consists of two tightly coupled components. The *data path* carries requests and model data between the network and the GPU, including packet processing, tensor transfers, and I/O movement. The *control path* governs how inference proceeds, including batching, scheduling, accelerator coordination, and state management during per-token decoding. SmartNICs are a natural target for offloading such functionality, as they lie on the critical path of request handling between the network and the GPU and bypassing the kernel. Prior SmartNIC-based systems successfully offload data-path tasks such as packet parsing, deep packet inspection, storage-client logic, and request handling to SmartNICs [17–19, 52], improving throughput for conventional one-shot inference workloads such as traditional Deep Neural Network (DNN) execution. In these workloads, each request executes as a largely self-contained forward pass, with scheduling decisions occurring at coarse granularity rather than at every token, resulting in a relatively lightweight control path with limited CPU involvement.

Autoregressive LLMs shift the bottleneck to the control path. LLM serving differs fundamentally. Autoregressive decoding transforms inference into a long-lived, stateful process in which each generated token depends on previously produced state. Latency-sensitive operations such as KV-cache management, batching decisions, and token streaming are tightly coupled to per-token scheduling. As a result, the control path becomes part of the critical loop. Existing SmartNIC-based inference systems offload portions of request handling or data movement [19, 52], but they do not address autoregressive decoding. Token-by-token execution, placement, and flow control repeatedly interact with GPU-resident state, while scheduling and coordination remain CPU-centric. Consequently, the critical loop remains anchored in host-managed mechanisms, leaving the server CPU on the steady-state inference path. This shift makes removing the CPU fundamentally harder, as both control and data responsibilities must be restructured across the system. This fundamentally *challenges the traditional separation between data and control paths*: decisions that were previously coarse-grained and host-driven must now be made at token-level granularity, tightly coupled with GPU state.

Why CPU-free inference is hard. Achieving a CPU-free[1] inference stack that delivers production-grade performance and predictability raises several challenges.

First, *eliminating host-driven scheduling and control* is fundamentally difficult. Autoregressive decoding turns inference into a long-lived, stateful process with fine-grained, per-token decisions (*e.g.,* batching, KV-cache management, and token generation). These decisions must be executed without host intervention, while ensuring forward progress and fairness, and without allowing control logic to interfere with GPU resources needed for inference.

Second, the system must *coordinate thousands of concurrent requests* without a host scheduler. This requires managing queuing, backpressure, and request lifecycles (including early exits and failures) across mismatched timescales (*i.e.,* microsecond network events, millisecond GPU kernels, and second-scale request lifetimes) using distributed control across the NIC and GPU.

Third, removing the host CPU requires rethinking *transport and request handling*. Conventional TCP/IP stacks and host drivers are tightly coupled with system memory and the OS scheduler, making them difficult to bypass without redesigning the data plane, especially under the limited compute budget of NIC-resident processors.

Finally, enabling *efficient network-to-GPU* communication demands direct data movement. This requires integrating RDMA and peer-to-peer DMA (*e.g.,* GPUDirect RDMA) into the serving stack to eliminate redundant copies and CPU involvement, without sacrificing throughput, latency, or fault isolation.

Blink: removing the server's main CPU from the critical path. In this paper, we present Blink, a novel LLM inference system that addresses these challenges by removing the host CPU from the steady-state inference path and redistributing responsibilities across a GPU and a SmartNIC DPU. To *eliminate host-driven scheduling and control*, Blink replaces the traditional decode loop with CUDA persistent kernels that implement a GPU-resident control plane, handling continuous batching, iteration-level scheduling, and Paged KV-cache management without per-token host interaction. To *coordinate thousands of concurrent requests* without a central scheduler, this control plane operates over shared data structures, enabling fairness, backpressure, and forward progress entirely on the GPU. To *remove dependence on host-based transport* and request handling, Blink offloads protocol parsing and request admission to the SmartNIC, bypassing kernel-mediated interfaces. Finally, to *enable efficient data movement*, the system uses one-sided RDMA and shared ring buffers to move data directly between the network and GPU VRAM, eliminating copies through host memory and CPU mediation while maintaining high throughput and low latency.

Evaluation benefits. We compare Blink with three state-of-the-art production LLM serving systems (TensorRT-LLM (TRT), vLLM, and SGLang) across four models under both isolated and multi-tenant execution. Even in *isolated settings*, Blink outperforms all baselines, reducing pre-saturation P99 time-to-first-token (TTFT) by up to 8.47× and P99 time-per-output-token (TPOT) by up to 3.40×, and improving decode throughput by up to 2.1×. These gains stem from eliminating per-token host interaction via a GPU-resident control loop. Under *host multi-tenancy interference*, Blink 's latency and throughput remain stable (within experimental variance of isolated values), whereas all baselines degrade by one to two orders of magnitude. As a result, Blink sustains up to 6.46× higher decode throughput (tokens/s) and up to 4.87× higher request throughput (requests/s) under CPU contention. These improvements translate into up to 48.6% lower energy per token in isolation, and up to 70.7% under interference. We plan to open-source Blink upon publication.

Contributions. This paper makes four primary contributions:

- *Identifying control-path dependence as the key bottleneck in LLM inference.* We show that modern LLM serving systems rely on fine-grained, host-driven control, making them sensitive to CPU interference and limiting efficient resource sharing.
- *A CPU-free LLM inference architecture.* We introduce Blink, a new class of serving systems that removes the host CPU from the steady-state critical path by redistributing control and data responsibilities across a GPU and a SmartNIC DPU.
- *GPU-resident control via persistent kernels.* We design a GPU-resident scheduler that replaces the host-driven decode loop, enabling continuous batching, scheduling, and KV-cache management without per-token CPU interaction.
- *Strong performance and isolation benefits.* Blink outperforms state-of-the-art systems even in isolation (up to 2.1× higher throughput and 8.47× lower pre-saturation P99 TTFT), while maintaining stable performance under CPU interference where existing systems degrade by orders of magnitude.

2 Background and Motivation

2.1 The Host CPU as a Latency Bottleneck

In every mainstream LLM serving stack, the host CPU orchestrates each iteration of autoregressive decoding. Request admission, continuous batching, KV-cache block management, and CUDA kernel dispatch all execute on host threads. Even with CUDA Graphs [27] amortizing individual kernel launch costs, the scheduler must return to the host after every decode step to update batch membership, manage the KV-cache block table, and dispatch the next graph. For instance, a single vLLM instance uses a multi-process architecture with an API server, an engine core, and one GPU worker per GPU [48]; this footprint scales with the chosen parallelism strategy,

[1] CPU-free refers to removing the host CPU from the inference critical path.

causing the number of host processes to grow rapidly with the deployment configuration [47].

This tight host-device coupling means that any perturbation to the CPU, *i.e.,* preemption, cache eviction, or elevated page-walk latency, directly inflates the Inter-Token Latency (ITL). Additionally, the GPU idles while the host rebuilds the microarchitectural state, and because decoding is iterative, the penalty compounds across hundreds of output tokens. This feedback loop between host jitter and token-level latency is quantified in §2.2.

Recent optimizations reduce but do not eliminate host involvement. The severity of host-side scheduling overhead has not gone unnoticed. Profiling studies report that CPU scheduling can consume up to 50 % of end-to-end inference latency on fast accelerators [43]. In response, recent engine revisions (*e.g.,* vLLM's V1 engine architecture [50] and SGLang's overlapped scheduling [39]) pipeline host-side work with GPU execution, reducing the scheduling tax under normal operating conditions. These optimizations mitigate CPU involvement but they do not remove the CPU from the critical path. Under colocation, co-tenants evict shared LLC lines and TLB entries, inflating the CPU operations that the overlap is designed to hide. Once host-side work exceeds the GPU execution interval available to mask it, the excess latency surfaces directly in each token's generation time. As we quantify in §2.2, even moderate interference is sufficient to break this overlap, because any CPU work that remains on the critical path turns shared microarchitectural contention into per-token latency.

Colocation is the norm, not the exception. Production clusters aggressively pack workloads to chase high utilization [34]. Cluster schedulers routinely colocate latency-critical services with best-effort or batch jobs, relying on priorities, cgroups, and NUMA partitioning to manage interference. Operators have developed a mature toolkit for isolation (Last Level Cache (LLC) partitioning (Intel CAT), huge pages, and Dynamic Voltage and Frequency Scaling (DVFS) tuning), yet all of these mitigations assume that the latency-sensitive workload can tolerate *some* CPU involvement. For LLM inference, where the CPU participates in every token, even well-managed colocation introduces *per-token jitter* that accumulates into SLO violations at the tail.

2.2 Motivating Measurement: CPU Interference

We begin with a simple experiment to illustrate the impact of CPU interference before analyzing root causes in §3. We conduct controlled experiments using vLLM (v0.13) [20] serving Llama-3 8B [13] on an NVIDIA H100 GPU driven by a dual-socket Xeon Gold server (for details, see Table 5), with ShareGPT v3 [33] conversation traces (mean input/output lengths of 1 019/463 tokens). CUDA Graphs are enabled in all configurations; hyper-threading and DVFS are disabled following standard practice for microarchitectural measurement. As the colocated interferer, we run `pbzip2` [11] compressing a large file, measuring both application-level SLO metrics and hardware performance counters via `perf stat`. The serving engine is fully warmed up before measurement begins; profiling spans the entire run including ramp-up and drain, which is conservative as lightly loaded transients dilute the steady-state interference effect.

Table 1 reveals the severity of the problem. Under interference from `pbzip2`, throughput drops by 3.8×, while P99 time-to-first-token (TTFT) inflates by up to 139×. Even moderate interference causes significant degradation, indicating that LLM serving performance is highly sensitive to CPU contention. This degradation is accompanied by substantial increases in cache misses (LLC), address translation misses (dTLB), and memory stalls, consistent with contention in shared CPU resources. A natural question is whether this fragility can be mitigated using standard datacenter techniques; §3 shows that such approaches are insufficient to eliminate the underlying CPU bottleneck.

Table 1. Impact of colocation on vLLM serving latency and microarchitectural counters. H100, Llama-3 8B, 7 req/s, w/ CUDA Graphs.

	Baseline	Interference	
		12×	24×
Throughput (tok/s)	7 475	4 554	1 961
Mean TTFT (ms)	73.7	4 865	16 552
P99 TTFT (ms)	150	6 366	20 959
Mean TPOT (ms)	13.0	13.6	14.8
P99 TPOT (ms)	14.4	18.0	32.1
P99 ITL (ms)	67.9	110.6	176.8
Instructions per cycle (IPC)	1.53	1.08	0.72
LLC miss rate (%)	7.0	43.2	71.6
LLC stall cycles	450 M	2 586 M	5 037 M
dTLB load misses	6 M	8 M	10 M
`walk_active`	383 M	920 M	1 454 M
CPU migrations	6	20	27

3 Limitations of CPU-Based Mitigations

We evaluate whether standard datacenter mitigations, such as huge pages, cache partitioning, CPU pinning, or dynamic core reallocation, can address the performance degradation observed in §2.2. In §3.1, we trace the root cause of the performance degradation using hardware performance counters and systematically evaluate each mitigation. In §3.2, we show that OS tuning is not a solution. All paths converge on the same irreducible overhead: the overhead of CPU-mediated orchestration itself. This finding motivates our new alternative architecture that entirely removes the host CPU from the steady-state inference loop (§3.3).

3.1 Microarchitectural Root Causes

Address translation intensifies LLC contention. Breaking down the LLC stall budget (from Table 1) exposes a two-stage amplification loop driven by address translation. In the baseline case, page-walk activity (*i.e.,* page-table traversal on a TLB miss, such as dTLB load misses and `walk_active`) accounts for 85 % of all LLC stall cycles, consistent with the translation-heavy behavior of Python-based serving stacks operating over fragmented virtual address spaces [21]. When interference is introduced, the total number of Translation Look-aside Buffer (TLB) misses rises only moderately (1.6×), but each miss becomes substantially more costly: the interferer's frequent memory-management operations (*e.g.,* `madvise`, `mprotect`, and `munmap`) induce TLB invalidations that force re-translation and concurrently contaminate the shared LLC with additional data and page-table metadata. Page walks that previously hit page-table entries residing in the LLC now must proceed all the way to DRAM, driving up `walk_active` cycles by 3.8×. In aggregate, LLC stall cycles increase by 11.2×, while the fraction of stalls attributed to page walks falls to under 25 %, not because page walks become cheaper, but because LLC data-access misses escalate even more sharply. This two-level amplification, where

TLB invalidations force page walks into an already-polluted LLC and drive up data-access misses, creates a cross-address-space interference mechanism that standard per-process mitigations cannot resolve.

Tail latency exhibits high variability. Even in isolation, host-mediated orchestration introduces jitter: individual token latencies spike due to batch scheduling, KV-cache management, and CUDA dispatch variance, even when per-request averages stay low. This is visible in the gap between P99 ITL and P99 time-per-output-token (TPOT) (67.9 ms vs. 14.4 ms, a 4.7× difference). Under interference, this jitter is amplified further: inter-kernel dispatch gaps widen from ≈1 µs to 40 µs (measured via NVIDIA Nsight Systems [32]), compounding across hundreds of output tokens per request.

Takeaway: *CPU interference degrades LLM serving severely: TLB invalidations and LLC pollution share the same microarchitectural resources, amplifying each other's impact.*

3.2 OS Tuning Is Insufficient

A natural question is whether standard operating system and hardware mitigations (*e.g.,* larger pages, cache partitioning, CPU affinity, and scheduling priorities) can restore performance under colocation. We use each mitigation as a controlled variable to systematically isolate the root cause of the degradation. The results reveal that interference enters through multiple microarchitectural channels, but the dominant bottleneck is not any single channel—it is CPU involvement on the critical path itself.

Unless otherwise noted, the experiments in this subsection use the same LLM server, the same H100 server, and CUDA Graphs enabled as described in §2.2, but with the interferer limited to 24 threads and 128 requests at 7 req/s. Additionally, we use a synthetic workload (random input & output lengths of 1 024 & 512 tokens) to maximise batch occupancy and stress the host scheduling path.

Huge pages (victim and interferer): negligible effect. Larger pages reduce TLB pressure by covering more memory per entry, potentially reducing the page-walk overhead identified in § 3.1. Table 2 shows that neither 2 MB pages for vLLM nor 1 GB gigantic pages for the interferer restore isolation-level performance. With 2 MB pages, all latency and throughput metrics remain within measurement noise; Data Translation Look-aside Buffer (dTLB) load misses drop only 16 %, consistent with the limited TLB reach benefit of 2 MB pages for Python-dominated working sets whose host-side footprint is dominated by small objects and metadata. Allocating the interferer's working set from a 1 GB hugepage pool does not help: P99 ITL *worsens* and LLC miss rates are unchanged, because gigantic pages reduce the interferer's own TLB misses but not its LLC footprint, similarly to what we observed in §3.1.

Core pinning: effective but impractical at scale. Dedicating physical cores with core pinning eliminates CPU migrations and prevents direct preemption by co-tenants, making it the strongest single-mechanism mitigation we evaluate. Following NVIDIA's Certified Systems Configuration Guide mandate of a minimum of six physical cores per GPU [31], we pin vLLM to cores 0–5 and confine the `pbzip2` interferer to the remaining cores. To stress-test pinning under realistic conditions, this experiment uses production-representative traffic: ShareGPT conversations with Poisson arrivals at 12 req/s measured over a 60 s window (unlike the micro-benchmarks above, which maximise batch occupancy). Table 3 shows the results.

Table 2. Ablation: page size and gigantic pages do not restore isolation (H100, CUDA Graphs enabled, 128 requests at 7 req/s). All runs include interference. Isolation baseline: 7 697 tok/s throughput, 13.5 ms mean TPOT, 5.9 % LLC miss rate.

	4 KB pages	2 MB pages	1 GB (interferer)
Throughput (tok/s)	4 813	5 053	4 540–5 298
P50 TTFT (ms)	12 276	12 612	12 578–15 205
P99 TTFT (ms)	29 208	26 780	29 191–32 270
P50 TPOT (ms)	19.8	19.9	17.9–19.6
P99 TPOT (ms)	25.0	25.4	27.2–30.5
P99 ITL (ms)	70.1	72.2	73.2–84.8
LLC miss rate (%)	71.3	71.2	68.5–70.3
dTLB load misses	8.8 M	7.4 M	7.8–10.0 M
`walk_active`	1 132 M	985 M	779–970 M

Table 3. Core pinning (6 dedicated cores) under `pbzip2` interference. ShareGPT workload, Poisson arrivals at 12 req/s, 60 s window.

	Isolation	Interference	Δ%
Total completed requests	502	415	-17.3 %
Mean throughput (tok/s)	4 130.50	3 457.85	-16.3 %
Mean throughput (req/s)	8.37	6.92	-17.3 %
P50 TTFT (ms)	10 052.08	12 539.18	+24.7 %
P99 TTFT (ms)	22 409.94	23 968.67	+7.0 %
P99.9 TTFT (ms)	23 925.64	25 732.75	+7.6 %
P50 TPOT (ms)	29.42	37.90	+28.8 %
P99 TPOT (ms)	162.57	192.45	+18.4 %
P99.9 TPOT (ms)	499.64	641.10	+28.3 %
P50 ITL (ms)	10.56	12.87	+21.9 %
P99 ITL (ms)	13.83	16.48	+19.2 %
P99.9 ITL (ms)	16.45	21.44	+30.3 %
Mean prefill throughput (tok/s)	1 375.08	1 223.27	-11.0 %
Median prefill throughput (tok/s)	161.87	116.05	-28.3 %
Mean decode throughput (tok/s)	2 780.60	2 274.77	-18.2 %
Median decode throughput (tok/s)	2 189.09	1 778.00	-18.8 %

We pin vLLM to six physical cores on the GPU-local socket (*i.e.,* 6 of 24 cores per socket, or 25 %). Even then, interference causes significant degradation across all metrics (Table 3): tail latency inflates by up to 30.3 % (P99.9 ITL and TPOT), decode throughput drops by 18.2 %, and the server completes 17.3 % fewer requests in the same window. The reason is structural: pinning removes scheduler contention, but the LLC, memory bandwidth, and socket interconnect remain shared with the co-located workload. Moreover, dedicating a minimum of six physical CPU cores per GPU [31] for a dense 8×H100 node, requires 48 dedicated physical cores—consuming all cores on our dual-socket Xeon server (24 cores per socket) and 38 % of even a high-end dual-socket EPYC platform (64 cores per socket). These dedicated cores sit largely idle between decode iterations, yet cannot be reclaimed for other tenants without re-introducing scheduling interference. In effect, core pinning converts a shared machine back into a dedicated one [34], eliminating the economic rationale for colocation; thus, core pinning is impractical at scale.

Hardware cache partitioning: eliminates LLC contention but not the latency overheads. Intel Cache Allocation Technology (CAT) requires core pinning as a prerequisite (a Class of Service (CLOS) is assigned to the *core*, not the process, and is overridden on context switches), so we evaluate whether adding cache partitioning provides additional benefit on top of pinning. We incrementally

allocate from 1 to 12 LLC cache ways to vLLM's pinned cores under interference. Table 4 summarizes the results.

Table 4. Effect of CAT cache-way allocation under interference (H100, CUDA Graphs, 128 requests at 7 req/s, dedicated CPU cores). LLC miss rate drops 8.5×, yet P99 ITL is virtually unchanged.

Cache ways	1	3	5	7	12
LLC miss rate (%)	57.6	26.6	11.1	7.0	6.8
IPC	1.16	1.31	1.48	1.52	1.55
LLC stall cycles	3 169 M	1 750 M	741 M	428 M	442 M
dTLB load misses	7.4 M	7.1 M	7.1 M	7.0 M	7.0 M
`walk_active`	895 M	596 M	462 M	420 M	400 M
P99 TTFT (ms)	29 675	26 627	27 146	26 352	26 157
P99 TPOT (ms)	23.3	22.6	21.3	20.9	21.3
P99 ITL (ms)	55.6	55.6	53.4	54.4	54.0

Allocating enough cache ways to vLLM recovers the LLC miss rate to near-isolated levels and significantly reduces LLC stall cycles. The mechanism is indirect: CAT does not partition the TLB, so the number of dTLB misses stays constant across all configurations (≈7 M). However, when a TLB miss occurs, the CPU must walk the page table to resolve the physical address, and those page-table entries themselves reside in the LLC. With more cache ways allocated to vLLM, page-table entries are more likely to remain LLC-resident, making each walk shorter.At 7 ways, the LLC miss rate reaches 7.0 % (vs. 6.8 % with all 12 ways) and LLC stall cycles drop 7.4× 7.4×.

Despite fully eliminating LLC contention, *tail latency is virtually unchanged*: P99 ITL ranges from 53.4 ms–55.6 ms across all configurations, a spread of less than 4 %. Hardware cache partitioning does not improve latencies because the dominant overhead, *i.e.*, host-side scheduling jitter and Compute Unified Device Architecture (CUDA) dispatch, *i.e.*, is unaffected by cache allocation (see §3.3.

Moreover, achieving this LLC recovery comes at a steep cost to co-tenants. Our server has 12 LLC ways; dedicating 7 to vLLM leaves only 5 ways for all remaining workloads on the socket, throttling their cache capacity by 2.4×. In a multi-GPU node, each GPU's serving process would need its own 7-way partition, quickly exhausting the available ways, making the mechanism simply unscalable. We also note that scheduling priority (`nice -20`) likewise has no measurable effect.

Takeaway: *Even fully eliminating LLC contention does not restore tail latency: host-side scheduling jitter and CUDA dispatch overhead remain on the critical path regardless of cache allocation.*

Practical limitations of RDT. Intel Resource Director Technology (RDT) [16], of which CAT is a component, is also impractical as an operator-level isolation mechanism: CLOS entries are scarce (9–15 per CPU) and must be contiguous; CLOS assignments are not preserved across context switches — when another process is scheduled on the same core, its CLOS overrides the previous one — so maintaining isolation requires dedicated cores or kernel-level modifications to save and restore CLOS state [42]; CAT addresses only cache occupancy, not memory bandwidth; and RDT is Intel-specific, with no equivalent on AMD or ARM platforms. It is a low-level mechanism, not a turnkey isolation toolkit.

Host-side orchestration jitter is directly observable. To confirm that host-side overhead is the residual bottleneck, we profile a single transformer layer's `forward()` with PyTorch [36]. Under interference, every host-side operation inflates: attention dispatch by 104 %, `cudaLaunchKernel` by 115 %, KV-cache dispatch (`reshape_and_cache_flash`) by 172 %, and metadata operations such as `aten::empty` by 81 %. Crucially, GPU-side kernel execution times are unchanged (*e.g.,* matmul remains at 0.41 ms–0.42 ms per call), confirming that the accelerator is simply *starved* by its host.

3.3 Implications: The Case for Host Removal

Our analysis reveals a fundamental architectural mismatch: today's serving systems place a fragile, interference-sensitive CPU on the critical path of every token, in an environment where interference is pervasive. Every mitigation we tested either had negligible effect or left a persistent performance residual. These are *not* implementation bugs; they reflect a structural mismatch between the iterative host–device coupling of current serving stacks and the shared-resource reality of multi-tenant datacenters.

Resource partitioning does not resolve the architectural mismatch. An operator might combine all preceding mitigations: dedicate physical cores via `cpuset`, partition LLC ways via CAT, and disable hyperthreading *i.e.,* approximating hardware-level socket partitioning—but at a prohibitive cost. Dynamic core reallocation systems such as Caladan [9] and Shenango [34] adjust core assignments at microsecond granularity, but optimize *how much CPU capacity* a tenant receives, not *whether the CPU is on the critical path.* LLC and memory-bus bandwidth are socket-wide resources that per-core reallocation does not effectively partition, and both systems require applications to link against custom userspace threading and networking runtimes in place of standard POSIX APIs—a requirement incompatible with the Python/PyTorch stacks underlying all major LLM serving engines.

Takeaway: *Achieving interference immunity requires removing the host CPU entirely from the steady-state inference loop.*

4 Blink Design

4.1 Overview

The design principle behind Blink is to make the host CPU a *provisioning plane* rather than a *data plane*: the host loads the model and captures CUDA graphs once at startup, then exits the inference path entirely. Blink is an end-to-end LLM inference serving system with five core goals: (1) **high concurrency with predictable latency**; (2) **CPU-free steady state** — after one-time initialization, host CPU is off the inference path; (3) **zero-copy prompt and token movement** between the network fabric and GPU memory via one-sided RDMA; (4) **compatibility with existing models** without requiring any model modification; and (5) **API compatibility** with OpenAI-style HTTP endpoints and Server-Sent Events (SSE) streaming semantics, enabling drop-in deployment.

Fig. 2 shows the high-level architecture and traces the lifecycle of prompts through the inference path (❶–⓭). Blink comprises two runtime planes. The **frontend** runs on the DPU's ARM cores: it receives client requests ❶, tokenizes prompts ❷, locates a free ring buffer slot ❸❹, writes prompts into GPU memory via one-sided RDMA ❺, retrieves generated tokens ⓫, detokenizes them ⓬, and streams responses back to clients ⓭. The **backend** runs entirely on the GPU: a persistent scheduler kernel polls a shared ring buffer for incoming prompts ❻, selects and launches pre-captured CUDA graphs ❼❽, computes tokens from the logits via on-device

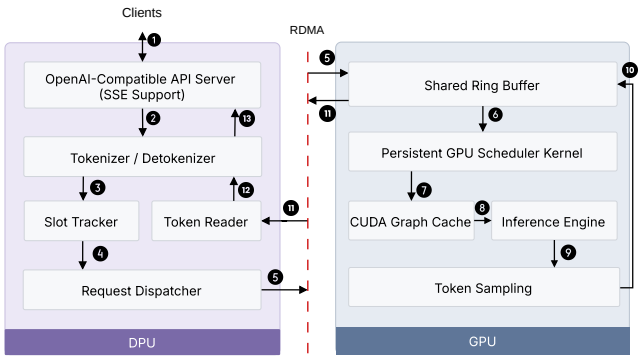


Figure 2. Architecture of Blink. After one-time initialization, the host CPU is idle; the steady-state inference path involves only the GPU and the DPU. Filled circles trace the lifecycle of a single request.

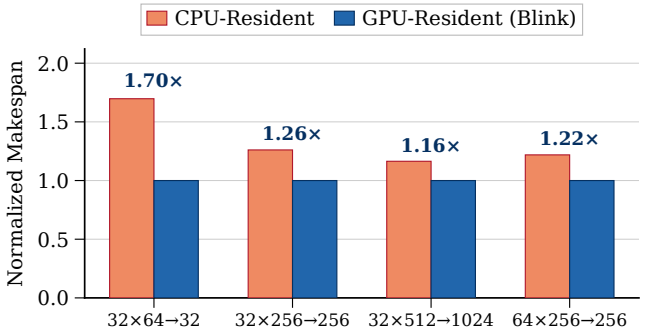


Figure 3. Normalized makespan: GPU-resident vs. CPU-resident scheduling on Qwen3-32B (N×I→O denotes N requests, I input tokens, O output tokens).

sampling ❾, and publishes results back to the ring buffer ❿—all without returning control to the host. The two planes communicate exclusively through a *GPU-resident ring buffer* accessed by RDMA (❺, ⓫), which serves as the sole rendezvous point and eliminates any need for host-mediated coordination. This cleanly separates *initialization* (host-assisted) from *steady state* (DPU+GPU only).

Why a DPU–GPU split? Modern SmartNICs outperform server-class CPUs (Xeons, EPYCs) in network bandwidth per core by 15–24× and memory bandwidth per core by 1.6–2.1× [35], making them well suited to transport and request management tasks. However, their limited core count cannot absorb the volume of per-iteration scheduling decisions in continuous batching without becoming a bottleneck. Blink exploits this asymmetry: the DPU handles request management and network I/O, while GPU-resident persistent kernels handle the latency-critical scheduling and execution loop. This split removes the host CPU from the data path without imposing DPU-side bottlenecks.

4.2 Backend: GPU-Resident Inference Engine

The GPU integrates four subsystems: **(1) a persistent GPU scheduler** that continuously manages request lifecycles, **(2) device-side CUDA graph launch** that avoids host-mediated kernel dispatch, **(3) a continuous batching engine** that admits new requests into running decode batches, and **(4) a lock-free ring buffer** for DPU–GPU coordination.

Persistent scheduler. Blink replaces the host-driven decode loop (§2.1) with a single persistent CUDA kernel that occupies one thread block (256 threads) and runs indefinitely. The scheduler executes an infinite control loop: (1) it scans the ring buffer for newly submitted prompts, (2) claims them via atomic compare-and-swap, (3) selects and launches the appropriate CUDA graph for prefill or decode, (4) polls device-resident output buffers for completion after token sampling, and (5) publishes generated tokens and status updates back to the ring buffer. Since the scheduler never yields to the host, there is no kernel launch overhead, no host–device synchronization, and no CPU-mediated bookkeeping on the token-critical path.

Parallel slot scanning. To avoid serialization, all 256 threads in the scheduler's thread block scan disjoint contiguous ranges of the ring buffer's slots in parallel and claim pending slots via atomic CAS, eliminating locks and inter-block synchronization. A complete scan of all 4096 slots completes in 1 µs–5 µs, ensuring that prompt admission latency is dominated by RDMA transfer time rather than scheduling overhead.

Fig. 3 quantifies the benefit of GPU-resident scheduling. We run identical workloads on the same compiled engine (Qwen3-32B, FP16, H100, batch size 16) under two scheduler placements that share an identical scheduling policy. In the CPU-resident baseline, the sampled tokens are copied to host memory over PCIe after each decode step, and the batch is reassembled on the CPU before launching the next graph; in contrast, the GPU-resident scheduler eliminates this per-step synchronization. In both cases, token sampling is performed on the GPU to best match popular CPU-centric systems such as vLLM. Across four workload configurations, the CPU path inflates the total makespan by 1.16–1.70×, with the largest penalty on short-output workloads where per-step PCIe round-trip overhead dominates compute.

Device-side CUDA graph launch. A key enabler of CPU-free operation is *device-side graph launch* [28, 30]: the persistent scheduler, itself wrapped as a CUDA graph, enqueues pre-captured inference graphs for execution directly from the device. Blink uses the fire-and-forget launch mode, which completes in ≈2 µs across representative graph topologies (straight-line, fork-join, and parallel)—5–8× faster than host-side launch (11 µs–17 µs depending on topology) and 2.7× faster than tail launch (≈5.5 µs). Over hundreds of decode steps these differences compound: fire-and-forget saves 4.6 ms–7.7 ms per 512-token generation compared to host launch. More critically, host launch reintroduces the CPU onto the critical path, re-exposing inference to host-side interference.

Challenge: the 120-launch hard limit. Fire-and-forget imposes a fundamental constraint that is not widely documented: the CUDA runtime limits outstanding fire-and-forget launches from a single parent graph execution to 120 [29]; once this limit is reached, further launches produce undefined behavior. For an inference server that must generate tokens indefinitely, this is a critical obstacle—a single request with 512 output tokens would exhaust the launch budget well before completion. Falling back to host launch after 120 iterations would reintroduce the CPU onto the critical path, and using tail launch exclusively incurs 2.7× higher per-launch overhead that accumulates across hundreds of decode steps.

Solution: window-based tail-launch recovery. Blink resolves this constraint with a *window-based tail-launch recovery* mechanism. The scheduler maintains a monotonically increasing launch counter in shared memory. Upon reaching the 120-launch limit, it issues a

single tail launch that atomically replaces the current graph execution with a fresh instance. All state—ring buffer pointers, KV cache metadata, in-flight requests—resides in persistent GPU memory and survives graph re-instantiation; the new instance resets the counter to zero and resumes the scheduling loop from the same logical point. This design achieves three properties: (*i*) *near-optimal launch latency*—fire-and-forget is used for 120 of every 121 iterations, with a single tail launch amortized across the window (<0.03 µs overhead per decode step); (*ii*) *seamless state continuity*—all metadata, KV cache state, and ring buffer contents reside in persistent GPU memory that survives graph re-instantiation; and (*iii*) *unbounded generation*—no upper bound on the number of generated tokens.

Completion detection. Fire-and-forget launches provide no host-side completion callback—the parent kernel receives no notification when a child graph finishes. Blink performs *polling-based completion detection* entirely on the device: after the inference engine executes a forward pass and token sampling writes the result, the scheduler polls the designated output buffer until the extracted token appears. For prefill, the scheduler polls a token extraction buffer populated by the inference graph upon computing the first output token; buffer occupancy signals prefill completion. For decode, the scheduler polls per-step extraction buffers written by each decode graph. The polling cost is negligible (a few shared-memory loads per iteration) relative to inference compute, and it preserves the invariant that no host involvement occurs during steady state.

Continuous batching with inline prefill. Blink implements continuous batching [54] with FCFS scheduling, matching the policy used by TensorRT-LLM, vLLM [20], and SGLang [56], so that evaluation differences (§6) isolate the architectural benefit of removing the host CPU rather than a scheduling advantage. To admit new requests without stalling ongoing generation, the scheduler implements *pause-and-resume batching* with an overlapped ring scan: while a decode graph executes asynchronously, the scheduler's 256 threads scan the ring buffer in parallel for pending prompts, *hiding scan latency behind decode compute*. If candidates are found, the scheduler evaluates three conditions before pausing: (*i*) pending prefills detected during the overlapped scan, (*ii*) free batch-slot capacity (accounting for slots that will complete this step), and (*iii*) sufficient fire-and-forget launch-window headroom for the prefill graph plus resumed decode. When all three hold, the scheduler pauses in-flight decode requests after the current step, executes a prefill graph for the new arrivals, merges the newly prefilled requests into the decode batch, and restarts the decode loop—all within a single persistent kernel invocation, with no host round-trip. This ensures that new requests are admitted within one decode step's latency, bounding TTFT without sacrificing decode throughput.

Ring buffer. The ring buffer resides in GPU memory and is the only shared data structure between the DPU and GPU, serving as the target of one-sided RDMA reads/writes. It consists of a fixed set of slots plus shared arenas for input and generated tokens. Each slot records per-request metadata (*e.g.,* prompt identity, token counts, and generation progress) and offsets into the token arenas. The scheduler advances each slot through a lifecycle state machine—EMPTY → PREFILL_PENDING → PREFILL_PROCESSING → DECODE_PROCESSING → DECODE_COMPLETED → EMPTY—and uses a DECODE_PAUSED state to support preemption and continuous batching. Ownership and state transitions use atomic Compare-and-Swap (CAS). The layout and update rules are designed to avoid intermediate copying, tolerate benign races, and ensure that RDMA-visible updates become visible in the intended order via memory fences.

CUDA graph cache. Blink uses precompiled TensorRT [25] engines for inference. During initialization, the host captures inference computation as CUDA graphs for a dense grid of (batch size, sequence length) pairs and instantiates each for device-side launch so the persistent scheduler can invoke them directly. All graphs reuse a single set of device buffers allocated once for the maximum supported shape, avoiding per-graph memory duplication. Since each captured graph consumes only 2 MB–3 MB—orders of magnitude smaller than the hundreds of MBs typical of PyTorch-based CUDA graphs—a cache of 650–1000 graphs fits within 2 GB–4 GB in our evaluated models, enabling fine-grained shape matching without exhausting the memory budget and overhead of dynamic capturing.

At runtime, the scheduler selects the tightest-fitting prefill graph via a precomputed lookup table indexed by (batch, sequence length), achieving $O(1)$ selection with no per-step search; a maximum-shape fallback graph handles any combination not in the cache. Token sampling (Top-P with temperature) is captured *inside* each graph, so the entire forward pass from attention through next-token selection executes as a single device-side launch with no host round-trip.

4.3 Model Compatibility

Because the persistent scheduler treats the inference graph as an opaque computation—populating input tensors, launching the graph, and reading output buffers—Blink inherits TensorRT's broad model support without modification. Dense Transformers, Mixture-of-Experts (MoE) models, and encoder-decoder architectures require only a TensorRT compilation step; no changes to the scheduler, ring buffer, or RDMA datapath are needed. Both model-level innovations (*e.g.,* new attention mechanisms [3, 40], quantization schemes, activation functions) and serving-level optimizations (chunked prefill [2], prefix caching [56], speculative decoding [23]) are orthogonal to Blink's architecture; we discuss concrete integration paths in §7.

4.4 Frontend: DPU-Side Control and Data Plane

The frontend runs on the BlueField DPU's ARM cores and manages the full request lifecycle from arrival through token delivery. A *request tracker* maintains per-request state—slot assignment, token counts, and completion status—coordinating prompt submission, token retrieval, and client-facing streaming across the subsystems described below. The frontend also hosts a thin OpenAI-compatible HTTP server with SSE streaming support; owing to the decoupled architecture, this interface layer could alternatively run on a separate gateway.

RDMA datapath. The frontend uses one-sided RDMA to directly read and write the GPU-resident ring buffer. It stages outgoing prompts and incoming results in separate DPU-local buffers to decouple submission from retrieval and to reduce interference between control metadata and bulk token traffic. On submission, the frontend transfers the tokenized prompt into the assigned slot and updates its state so the backend can begin prefill. When requests arrive in bursts, the frontend coalesces transfers to amortize RDMA overhead across multiple prompts.

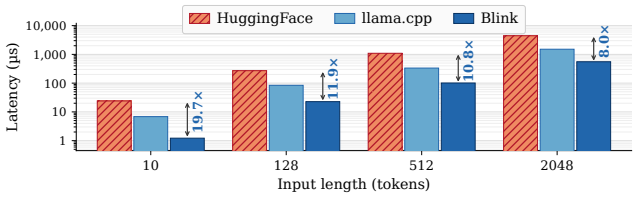


Figure 4. Tokenization latency on BlueField-3 ARM A78 cores (Blink) vs. Intel Xeon (HuggingFace, llama.cpp). Labels show speedup relative to HuggingFace.

Slot tracker. Rather than scanning all ring buffer slots via RDMA before each submission, the slot tracker maintains a local availability cache on the DPU, refreshed periodically via a single bulk RDMA read. A hint-based circular scan finds empty slots in $O(1)$ amortized time, improving spatial locality and reducing per-submission RDMA overhead.

Token reader. A background token reader continuously polls the ring buffer for generated tokens. Each cycle, it issues one RDMA read to refresh cached slot metadata (64 KB), then compares each active slot's generation count with its local state to detect new output. To minimize TTFT, new slots go to an *urgent slot* scanned first, so the first token is retrieved within one poll interval. Adaptive polling bounds per-token latency while limiting RDMA traffic. Under bursty arrivals, the reader caps per-poll work and uses large RDMA task pools to avoid completion-queue saturation; a dedicated progress thread processes completions to sustain throughput. Retrieved tokens go to the detokenizer and are streamed to clients via SSE.

Tokenizer. Tokenization on the DPU must be efficient on the ARM microarchitecture. Blink implements a tokenizer with merge rules in a 64-byte-aligned flat hash table, packing four key-value pairs per L1D cache line; 16-byte-aligned symbol nodes keep short-word working sets within two cache lines, avoiding heap indirection. Regex pre-tokenization uses ARM NEON SIMD for byte classification at 16 bytes per cycle, and all per-request state lives in pre-allocated thread-local buffers, eliminating heap allocation on the request path. Fig. 4 compares Blink's tokenizer on BlueField-3 ARM A78 cores with HuggingFace's (used by vLLM and SGLang) and llama.cpp's [10] tokenizers on an Intel Xeon CPU. Despite the ARM cores' lower clock speed, Blink's tokenizer is 8–19.7× faster than HuggingFace for inputs of 10–2 048 tokens and consistently outperforms llama.cpp, showing that the DPU introduces no tokenization bottleneck.

5 Implementation

Blink's GPU-resident backend comprises approximately 16 k lines of CUDA and C++ code implementing the persistent scheduler, device-side graph launch mechanism, continuous batching engine, KV-cache management, and token sampling, targeting CUDA 13.1 and leveraging the TensorRT inference engine for pre-captured model graphs, while the DPU-resident control and data plane adds approximately 17 k lines of C/C++ code implementing HTTP parsing and request validation, RDMA orchestration via the DOCA 3.2 SDK, ring buffer coordination, and SSE streaming, with the DPU also providing optional tokenization on its ARM cores to offload this traditionally host-side work and reduce end-to-end latency.

6 Evaluation

We evaluate Blink against three state-of-the-art LLM inference serving systems across four models, under both isolated and co-located multi-tenant conditions. We address three questions:

Q1 How does Blink compare to existing inference systems in latency and throughput? (§6.2)

Q2 How well does Blink preserve performance under co-location with noisy tenants? (§6.3)

Q3 How does Blink's architecture affect energy efficiency under isolated and multi-tenant execution? (§6.4)

We report P99 TTFT, P99 TPOT, and throughput (completed req/s), emphasizing tail latencies as they determine whether SLOs can be met in production. Across all three questions, the pattern is consistent: in isolation, Blink provides the best pre-saturation latency envelope and the highest saturated throughput; under CPU interference, its operating range and plateau remain essentially unchanged while CPU-mediated baselines collapse; and because all systems draw similar wall power, these throughput gains translate directly into lower energy per token.

6.1 Experimental Setup

Testbed. Table 5 lists the hardware. The Blink backend runs on this server; its frontend runs on a separate BlueField-3 DPU machine, connected via a 200 Gbps RDMA link (DOCA SDK v3.2.1). A workload generator reaches the frontend through a 10 Gbps switch. Baselines run on the same server with the workload generator connected via the same switch.

Table 5. Hardware configuration.

Component	Specification
GPU	NVIDIA H100 (96 GB HBM3)
CPU	2 × Intel Xeon Gold 6336Y (96 cores @ 2.40 GHz), DVFS disabled, governor: `performance`
DRAM	256 GB DDR5
Network	ConnectX-6 (200 Gbps)
DPU	BlueField-3 (16 ARM Cortex-A78, 32 GB)
OS	Linux 5.15 (Ubuntu 22.04 LTS)

Models and workloads. We evaluate four models that span a range of parameter counts and architectural choices: Llama-3 8B [13] (dense, 8B parameters), Phi-4 15B [1] (dense, 14.7B parameters), Qwen-3 32B [53] (dense, 32B parameters), and Qwen-3 30B-A3B [53] (MoE, 30B total/3B active parameters). All experiments use paged attention and fp16 precision. We use the *guidellm* tool [49] configured with the ShareGPT v3 dataset [33], which provides naturalistic conversation traces. For each experiment we sweep over 13 offered load levels from 1 to 32 requests/second, measuring performance at each rate for 60 s before advancing to the next. Both Blink and TensorRT-LLM use fp16 precision with paged KV cache, float16 attention (context FMHA), fused MLP, and removed input padding; the MoE plugin is enabled (fp16) only for Qwen-3 30B-A3B.

Baselines. We compare against three production-grade systems: (1) *TensorRT-LLM v1.1.0* [26] with TensorRT v10.14 [25] as its execution backend—using the *same* TensorRT engines as Blink to isolate the effect of CPU-mediated orchestration; (2) *vLLM v0.13.0* [20], the widely deployed open-source serving system; and (3) *SGLang v0.5.8* [56], a recent high-performance serving system with RadixAttention. All baselines use recommended production settings with

Table 6. Pre-saturation summary over the Blink-defined operating range. Repeated runs are averaged per load to match the figures. TTFT and TPOT are in milliseconds; Tput@sat is achieved throughput at Blink's saturation point. Best values per model are boldfaced.

Model	System	Geo. P99 TTFT	Geo. P99 TPOT	Tput at sat.
Llama-3 8B $\lambda \leq 12$	Blink	**653.8**	**15.1**	**11.87**
	TRT-LLM	880.0	17.7	10.80
	vLLM	1309.6	24.2	9.12
	SGLang	1747.1	30.7	7.88
Phi-4 15B $\lambda \leq 7$	Blink	**1109.4**	**25.0**	**6.72**
	TRT-LLM	1453.8	29.8	6.42
	vLLM	1683.7	34.5	6.05
	SGLang	2874.1	47.9	5.58
Qwen-3 32B $\lambda \leq 2$	Blink	**9481.3**	**113.4**	**2.00**
	TRT-LLM	9621.4	115.2	1.97
	vLLM	10862.4	133.7	1.88
	SGLang	11413.0	123.3	1.85
Qwen-3 30B-A3B $\lambda \leq 4$	Blink	**1397.5**	**35.5**	**4.85**
	TRT-LLM	4814.7	65.8	3.61
	vLLM	8919.2	90.9	2.91
	SGLang	11839.8	120.8	2.62

CUDA Graphs; chunked prefill, prefix caching, and CPU offloading are disabled for controlled comparison, as Blink does not yet incorporate these optimizations.

Interference workloads. To model multi-tenant "noisy neighbor" scenarios, we colocate two CPU-intensive workloads with inference: *pbzip2* [11] compressing a 50 GB file with 45 threads, and *Ninja* [24] building the LLVM [22] compiler source with 45 parallel jobs. Following NVIDIA's guidelines [31], we reserve six cores for inference and leave the remaining 90 host cores to the interferers. We also tested ffmpeg video encoding and observed qualitatively similar results, so we omit those experiments for brevity.

6.2 Isolated Performance (Q1)

We first evaluate under isolated execution which is the best-case scenario for CPU-mediated systems, where inference has exclusive access to host resources. Rather than centering the evaluation on a single unloaded rate, we summarize performance over the *Non saturated operational range*: for each model, the set of offered loads up to Blink's throughput saturation point, where the throughput curve transitions from linear growth to a stable plateau. This asks the deployment question that matters: over the full load range that Blink can still absorb before entering saturation, how do competing systems behave? This framing is intentional rather than favorable: if a baseline already queues inside the load interval that Blink can still sustain, that is precisely the deployment disadvantage the paper aims to measure.

We identify the saturation point from the same rate-averaged throughput curves in Fig. 7(a–d) using a two-segment fit (linear growth followed by plateau). To match the figures, we first average repeated runs at each offered load and then aggregate across loads. The resulting Blink operating ranges, where λ denotes the offered request rate in req/s, are $\lambda \leq 12$ for Llama-3 8B, $\lambda \leq 7$ for Phi-4 15B, $\lambda \leq 2$ for Qwen-3 32B, and $\lambda \leq 4$ for Qwen-3 30B-A3B. Table 6 reports geometric-mean P99 TTFT and P99 TPOT over the corresponding rate-averaged latency curves in Figs. 6(a–d) and 6(e–h), together with throughput at the offered load equal to Blink's saturation point from Fig. 7(a–d).

Within Blink's serviceable range, Blink provides the best latency envelope. Table 6 shows that across the entire load range Blink can sustain before saturation, Blink delivers the lowest geometric-mean P99 TTFT and TPOT on three of the four models and is near-parity with TensorRT-LLM on the strongly GPU-bound Qwen-3 32B. On Llama-3 8B, baselines incur 1.35–2.67× higher geometric-mean P99 TTFT and 1.17–2.03× higher geometric-mean P99 TPOT. On Phi-4 15B, the corresponding gaps are 1.31–2.59× for TTFT and 1.19–1.92× for TPOT. The MoE result is especially strong: on Qwen-3 30B-A3B, baselines incur 3.45–8.47× higher TTFT and 1.85–3.40× higher TPOT than Blink. Throughput at saturation makes the serviceability difference explicit: Blink sustains 11.87 req/s versus 10.80 req/s for TensorRT-LLM on Llama-3 8B, 6.72 req/s versus 6.42 req/s on Phi-4 15B, and 4.85 req/s versus 3.61 req/s on the MoE model.

Two architectural properties account for the TTFT advantage. First, the persistent GPU scheduler continuously scans the ring buffer with 256 threads in parallel and can claim a newly submitted prompt within 1 µs–5 µs (§4.2), whereas CPU-mediated systems incur host-side scheduling and kernel dispatch delays of tens to hundreds of microseconds per step. Second, prompt tokens reach GPU memory via zero-copy one-sided RDMA from the DPU, bypassing the host memory hierarchy entirely; in contrast, all baselines copy input tokens through host DRAM before transferring them to the GPU. Together, these properties reduce the time between prompt arrival and the start of prefill computation, which is the dominant contributor to TTFT at pre-saturation loads.

The same-engine comparison isolates the orchestration benefit. The comparison to TensorRT-LLM is especially clean because both systems use the same TensorRT engines. Across Blink's operating range, Blink reduces geometric-mean TTFT by 1.35× on Llama-3 8B, 1.31× on Phi-4 15B, and 3.45× on Qwen-3 30B-A3B. These gains therefore come from removing host-mediated orchestration rather than changing the model backend. Qwen-3 32B is the expected limiting case: its operating range is narrow and the P99 gap compresses, confirming that once the workload becomes predominantly GPU-bound there is less host-side latency left to remove. However, the advantage re-emerges at deeper tail percentiles: Fig. 5 plots P99.9 TTFT and TPOT for all four systems on Qwen-3 32B. At P99.9, baselines incur 4 %–8 % higher TTFT and 15 %–48 % higher TPOT than Blink across saturated loads—a clear separation that P99 averages mask. Supplementary materials contain corresponding P99.9 breakdowns for all four models.

Beyond saturation, Blink sustains the highest plateau. The throughput curves in Fig. 7(a–d) show that Blink also reaches the latest or tied-latest saturation point and then sustains the highest plateau throughput on every model. The plateau is 11.96 req/s for Llama-3 8B, 6.86 req/s for Phi-4 15B, 2.13 req/s for Qwen-3 32B, and 5.07 req/s for Qwen-3 30B-A3B. Relative to TensorRT-LLM, this corresponds to 9 %, 6 %, 8 %, and 37 % higher plateau throughput, respectively. The MoE model consistently exhibits the largest gain across all metrics; two reinforcing factors explain why.

Why the MoE advantage is amplified. First, MoE models have an unfavorable *compute-to-orchestration ratio*: Qwen-3 30B-A3B activates only 3 B of its 30 B parameters per token, so each decode step completes quickly on the GPU—comparable in compute to a small dense model—yet the per-step CPU orchestration cost

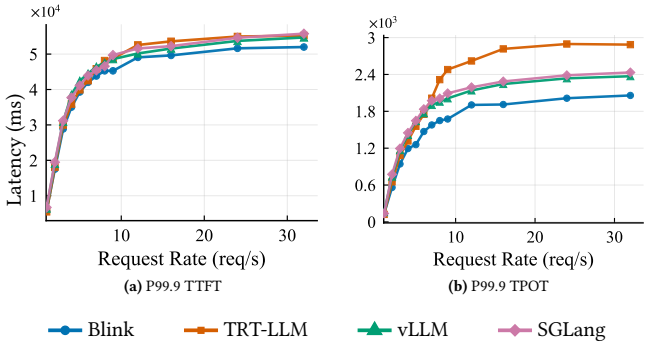


Figure 5. P99.9 tail latency for Qwen-3 32B (isolated). Although P99 latencies are near-parity, P99.9 reveals a consistent Blink advantage across all baselines that grows with load.

(scheduler iteration, host–device synchronization, batch reassembly) remains constant. CPU overhead therefore consumes a much larger fraction of total step time than in dense models where GPU compute dominates, and removing that overhead yields a proportionally larger speedup. This explains the progression from 8 % on Qwen-3 32B (GPU-bound; CPU overhead is a small fraction) to 37 % on Qwen-3 30B-A3B (fast active compute; CPU overhead is a large fraction). Second, MoE expert routing is *data-dependent but not shape-dependent*: which experts are selected varies with each token's hidden state, but all tensor dimensions remain fixed regardless of the routing decision. TensorRT's MoE plugin handles gating, token-to-expert dispatch, and gather internally within fixed-size buffers, so the entire forward pass—including MoE layers—is captured as a single CUDA graph with static shapes. Blink's device-side graph launch therefore executes MoE models without any host intervention to interpret router outputs or dynamically dispatch expert kernels. CPU-mediated baselines, by contrast, still interpose host-side scheduling on every decode step, paying the same per-step orchestration tax even though the compiled engine itself could run autonomously.

6.3 Performance Under CPU Interference (Q2)

Production deployments routinely co-locate inference with other tenants. We evaluate whether Blink's host-decoupled execution preserves the same operating range under severe CPU contention. We deliberately keep the ranges fixed to the isolated Blink saturation points from §6.2: this tests whether the load range that Blink unlocks in isolation remains usable under colocation, and whether competing systems can match that behavior. If a baseline cannot sustain that same range once interference appears, that loss of serviceable capacity is itself the result.

Blink's operating range survives interference intact. Across the full Blink-defined operating range, Blink remains effectively unchanged under interference: TTFT inflation stays within 0.92–1.14×, TPOT inflation within 0.97–1.04×, and throughput at the saturation point within 0.99–1.02× of isolation, showing that host CPU contention does not perturb the steady-state decode path.

CPU-coupled baselines incur a large interference tax inside Blink's range. Once we evaluate the same load ranges on CPU-coupled baselines, the contrast is sharp. On Llama-3 8B, baselines suffer 8.43–18.84× TTFT inflation, 5.77–11.10× TPOT inflation, and retain only 0.38–0.48× of their isolated throughput at saturation. On Phi-4 15B, the corresponding inflation is 3.82–10.66×

Table 7. Same pre-saturation summary as Table 6, but under CPU interference. Bracketed terms show interference / isolation; throughput brackets report retention at Blink's saturation point. Raw latencies are in milliseconds and throughput in requests/s. Best raw values per model are boldfaced.

Model	System	Geo. P99 TTFT	Geo. P99 TPOT	Tput at sat.
Llama-3 8B $\lambda \le 12$	Blink	**652.8 [1.00]**	**15.2 [1.00]**	**11.83 [1.00]**
	TRT-LLM	16574 [18.84]	196.6 [11.10]	4.13 [0.38]
	vLLM	14563 [11.12]	178.2 [7.35]	4.00 [0.44]
	SGLang	14728 [8.43]	177.3 [5.77]	3.78 [0.48]
Phi-4 15B $\lambda \le 7$	Blink	**1023.2 [0.92]**	**24.4 [0.98]**	**6.77 [1.01]**
	TRT-LLM	15496 [10.66]	183.7 [6.17]	2.62 [0.41]
	vLLM	12016 [7.14]	163.5 [4.74]	2.87 [0.47]
	SGLang	10991 [3.82]	150.6 [3.15]	2.62 [0.47]
Qwen-3 32B $\lambda \le 2$	Blink	**9415.6 [0.99]**	**117.8 [1.04]**	**2.05 [1.02]**
	TRT-LLM	16195 [1.68]	371.4 [3.23]	1.00 [0.51]
	vLLM	16702 [1.54]	353.0 [2.64]	1.20 [0.64]
	SGLang	18371 [1.61]	413.7 [3.35]	1.10 [0.59]
Qwen-3 30B-A3B $\lambda \le 4$	Blink	**1589.7 [1.14]**	**34.4 [0.97]**	**4.81 [0.99]**
	TRT-LLM	23587 [4.90]	604.9 [9.19]	1.01 [0.28]
	vLLM	17989 [2.02]	276.8 [3.04]	1.57 [0.54]
	SGLang	23449 [1.98]	478.1 [3.96]	1.18 [0.45]

for TTFT, 3.15–6.17× for TPOT, and 0.41–0.47× for throughput retention. Even on the GPU-bound Qwen-3 32B, where isolated gaps are smallest, interference still inflates TTFT by 1.54–1.68× and TPOT by 2.64–3.35×, while reducing throughput at saturation to 0.51–0.64× of isolation. The MoE model shows the clearest failure mode: within Blink's operating range, TensorRT-LLM incurs 4.90× TTFT inflation and 9.19× TPOT inflation, while its throughput at saturation collapses from 3.61 req/s to 1.01 req/s. This behavior is visible directly in Figs. 6(a–d), 6(e–h), and 7(a–d), where the baseline dashed curves separate sharply from their isolated counterparts.

After saturation, Blink's plateau is preserved while baselines collapse. Fig. 7(a–d) shows that Blink's saturation point is unchanged under interference on all four models, and its plateau throughput is preserved (99 %–100 % retention). In contrast, baseline plateau retention falls to 32 %–48 % on Llama-3 8B, 42 %–50 % on Phi-4 15B, 45 %–64 % on Qwen-3 32B, and 36 %–59 % on Qwen-3 30B-A3B. Under interference, Blink's plateau remains 3.18–3.38× higher than the baselines on Llama-3 8B, 2.41–2.50× higher on Phi-4 15B, 1.69–2.42× higher on Qwen-3 32B, and 2.91–4.08× higher on Qwen-3 30B-A3B.

6.4 Energy Efficiency (Q3)

We measure server-level wall power using a calibrated smart meter that records cumulative energy at 1 min intervals from server's PSU feed. For Blink, we additionally account for the BlueField-3 DPU's power consumption, sampled at 0.5 s intervals via the DPU's on-board meter. We report energy per token (mJ/tok) as the product of average wall power and benchmark duration, divided by the number of tokens successfully processed.

Isolation. Blink consumes 363 mJ/tok–1 306 mJ/tok across four models (Fig. 8(a)), 13.7 %–48.6 % less than the most efficient baseline per model. The gap ranges from Phi-4 (502 vs. 582 mJ/tok for vLLM, where model size limits relative scheduling overhead) to Qwen-3 30B-A3B (607 vs. 1 180 mJ/tok for TensorRT-LLM, where MoE expert routing amplifies CPU scheduling overhead). For Qwen-3

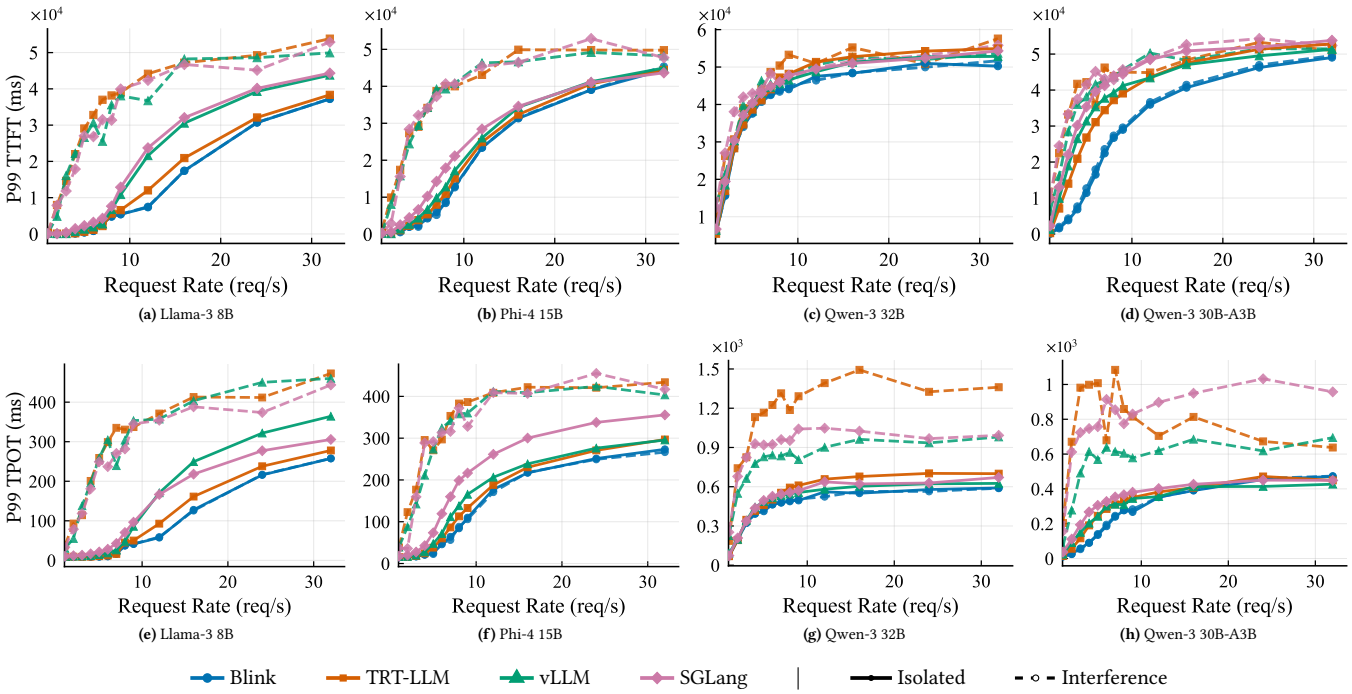


Figure 6. P99 tail latency across four models. Top row (a–d): TTFT; bottom row (e–h): TPOT. Solid lines show isolated execution; dashed lines show CPU interference. Within Blink's operating range, Blink maintains a flatter latency envelope and near-overlapping isolated and interference curves, whereas baseline systems queue earlier and suffer severe tail inflation under co-location.

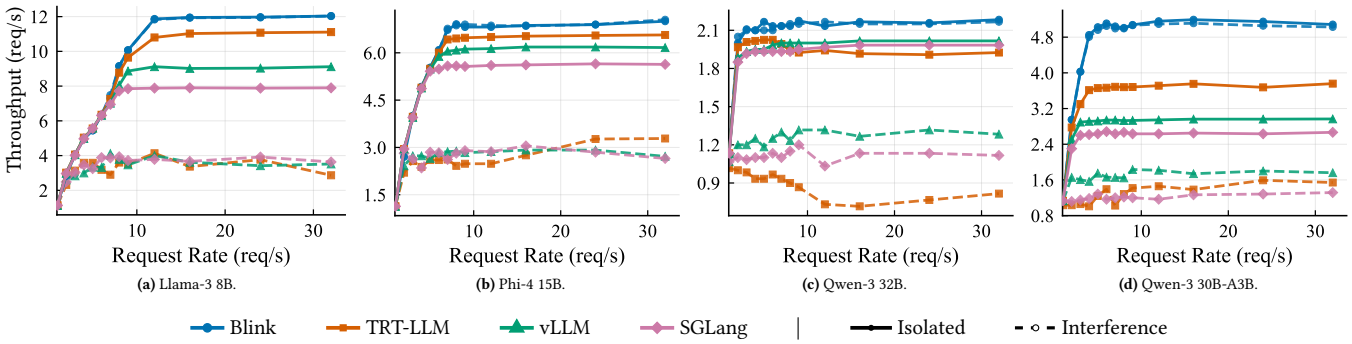


Figure 7. Throughput across four models. Solid lines show isolated execution; dashed lines show CPU interference. Blink reaches the latest or tied-latest saturation point in isolation and preserves its plateau under interference, while baselines either saturate earlier or collapse to much lower plateaus.

32B, the largest dense model, Blink achieves 1 306 mJ/tok versus 1 580 mJ/tok for SGLang—a 17.3 % reduction.

Interference. Under colocated interference (Fig. 8(b)), Blink achieves 423 mJ/tok–1 584 mJ/tok while CPU-mediated baselines rise to 1 045 mJ/tok–3 597 mJ/tok, a 41.4 %–70.7 % reduction relative to the best baseline per model. All four systems draw comparable wall power (1.1 kW–1.4 kW), so energy per token tracks inversely with throughput. When CPU contention collapses baseline throughput (§6.3) at constant power, their energy per token inflates by 69 %–182 %. Blink's overhead is at most 21 %: removing the host CPU from the critical path makes energy efficiency structurally independent of host contention.

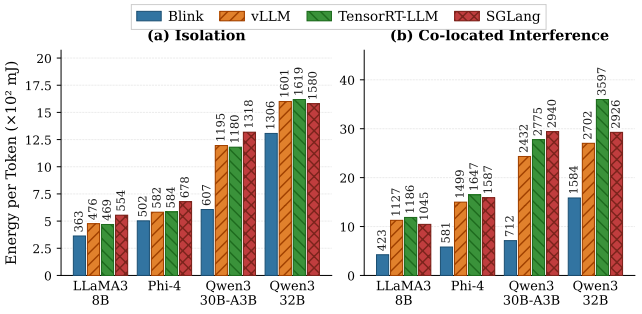


Figure 8. Energy per token under (a) isolation and (b) colocated interference.

7 Discussion

A natural concern is whether removing the host CPU precludes standard serving optimizations; we argue it does not, as Blink exposes clear extension points for each major technique.

Tensor parallelism and pipeline parallelism. While Blink's current prototype targets a single GPU, the same control-plane structure extends to multi-GPU deployments. Tensor-parallel sharding and pipeline parallelism can be realized by instantiating a persistent scheduler on each GPU (with its own local shared-memory region) and inserting GPU-native communication primitives (*e.g.,* NCCL collectives or GPU-initiated RDMA via IBGDA [55]) between graph executions; device-side synchronization enforces the required ordering. For MoE models, GPU-initiated all-to-all dispatch [55] is particularly attractive, as it avoids NCCL's CPU proxy overhead. Because these mechanisms execute on the GPUs, they preserve Blink's CPU-free critical path. A full implementation and evaluation of these multi-GPU extensions is left to future work.

Serving optimizations. Several widely studied optimizations map naturally onto Blink's GPU-resident scheduler. *Chunked prefill* [2]—the scheduler already tracks per-request progress and KV-cache status, enabling incremental prefill without changing the data plane. *Prefix caching* [56]—the paged KV cache provides reusable blocks; a GPU-resident trie or hash table can map token prefixes to KV-block ranges inside the scheduler. *Speculative decoding* [23]—the scheduler runs draft generation followed by target verification and publishes only accepted tokens to the ring buffer. *Disaggregated prefill/decode* [57]—separate scheduler instances for prefill and decode can transfer KV state over NVLink without involving the CPU. *KV cache offloading*—when HBM is exhausted, inactive KV blocks can be offloaded to a peer GPU via NVLink-based device-initiated transfers [46] or CUDA managed memory.

CPU/DRAM offloading. When models exceed GPU memory capacity, which is increasingly common with MoE architectures whose total parameter counts far exceed their active parameters, systems resort to CPU/DRAM offloading for expert weights [14, 15]. In this regime the host CPU is not only on the scheduling path but also on the data-movement path: it must orchestrate dynamic expert routing and migrate weights between host memory and GPU memory on every token. This makes the interference problem described in §3 strictly worse, as expert migration latency compounds with orchestration jitter and memory bus contention directly stalls expert fetches. Blink currently targets the common case where the served model fits in GPU memory. Extending the architecture to offloading scenarios is a natural next step; the DPU's RDMA engine could manage expert migration directly, keeping weight transfers off the host memory bus while preserving the CPU-free serving path.

Broader ecosystem trends. Blink's ARM-based DPU frontend is well-aligned with the growing role of ARM in AI datacenter infrastructure, exemplified by the Arm AGI CPU [5] co-developed with Meta. As ARM-based DPUs gain higher core counts and tighter RDMA integration, Blink's frontend stands to benefit directly.

8 Related Work

SmartNIC/DPU offload for accelerator services. The most closely related systems offload portions of ML or storage datapaths to SmartNICs, but all target either one-shot DNN inference or non-inference workloads. Lynx [45] proposes an accelerator-centric architecture where the SmartNIC replaces the host CPU as the orchestrator for GPU network services: it receives requests, dispatches them to GPUs via RDMA, and returns responses without host involvement. It operates at request-level granularity on one-shot workloads (face verification, LeNet) with no support for autoregressive and stateful LLMs. SplitRPC [19] steers inference payloads directly from the NIC to GPU memory, reducing host-side RPC data-movement overheads, yet retains the host CPU for orchestration, limiting it to stateless, one-shot DNN inference. Conspirator [52] moves an ML control plane onto SmartNIC ARM cores for job-level scheduling across GPU instances in distributed training, but the host CPU remains on the critical path for kernel invocation and result retrieval, and scheduling operates at job granularity rather than per-token. OS2G [17] offloads an object-storage client onto a DPU, introducing GPUDirect DPU for direct DPU-to-GPU transfers and fully bypassing the host for deep-learning training I/O, validating the same DPU-to-GPU transfer primitive Blink uses, but targeting storage rather than inference. DeepEP [55] uses GPU-initiated RDMA (IBGDA) to bypass NCCL's CPU proxy for MoE expert-parallel dispatch, but targets a single collective primitive within host-driven stacks rather than end-to-end serving.

LLM serving and GPU execution A large body of work optimizes the host-driven serving loop: continuous batching [54], chunked prefills [2], prefill/decode disaggregation [57], PagedAttention [20], and further scheduling and memory optimizations [4, 37, 41, 51]. On the GPU side, CUDA Graphs [27], device-side graph launch [28], memory-efficient attention [6, 7], and production runtimes such as TensorRT-LLM [26] accelerate individual operations. All retain the host CPU as the locus of scheduling and I/O; Blink instead runs these batching and memory policies in its GPU-resident scheduler, keeping the host off the steady-state critical path.

None of these systems addresses the challenges unique to autoregressive LLM inference: token-level continuous batching, stateful KV-cache management across long-lived decode sessions, and per-token streaming. Blink is the first to restructure the inference serving stack end to end, to eliminate the host CPU from the entire autoregressive inference lifecycle.

9 Conclusion

This paper introduced Blink, a CPU-free LLM serving architecture that removes the host CPU from the steady-state inference path by co-designing SmartNIC-resident ingress, zero-copy RDMA into GPU memory, and a GPU-resident persistent scheduler for continuous batching, KV-cache management, and token generation. Across four models spanning dense and MoE architectures (Llama-3 8B, Phi-4 15B, Qwen-3 32B, and Qwen-3 30B-A3B), Blink improves pre-saturation P99 TTFT by up to 8.47× and P99 TPOT by up to 3.40×, decode throughput by up to 2.1×, and energy per token by up to 48.6 % over TensorRT-LLM, vLLM, and SGLang, while maintaining performance under multi-tenant CPU contention where baselines degrade by one to two orders of magnitude.

More broadly, Blink shows that persistent accelerator-resident control combined with SmartNIC-resident I/O can replace host-mediated orchestration for latency-sensitive workloads, and that decoupling inference from shared host resources will be key to performance isolation and server consolidation as datacenters scale.

References

[1] Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, et al. 2024. Phi-4 Technical Report. (2024). arXiv:2412.08905 [cs.CL] https://arxiv.org/abs/2412.08905

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Santa Clara, CA.

[3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 4895–4901.

[4] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, et al. 2022. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[5] Arm. 2026. Arm AGI CPU. https://www.arm.com/products/cloud-datacenter/arm-agi-cpu.

[6] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*.

[7] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[8] Ina Fried. 2024. OpenAI says ChatGPT usage has doubled in the last year. https://www.axios.com/2024/08/29/openai-chatgpt-200-million-weekly-active-users.

[9] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. 2020. Caladan: Mitigating Interference at Microsecond Timescales. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association.

[10] Georgi Gerganov et al. 2023. llama.cpp: LLM Inference in C/C++. https://github.com/ggml-org/llama.cpp.

[11] Jeff Gilchrist. 2004. Parallel Data Compression with bzip2. In *Proceedings of the 16th IASTED International Conference on Parallel and Distributed Computing and Systems (PDCS)*. 559–564.

[12] Kanishk Goel, Jayashree Mohan, Nipun Kwatra, Ravi Shreyas Anupindi, and Ramachandran Ramjee. 2025. Niyama: Breaking the Silos of LLM Inference Serving. arXiv:2503.22562 [cs.LG] https://arxiv.org/abs/2503.22562

[13] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, et al. 2024. The Llama 3 Herd of Models. (2024). arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[14] En-Ming Huang, Li-Shang Lin, and Chun-Yi Lee. 2025. Efficient CPU-GPU Collaborative Inference for MoE-based LLMs on Memory-Limited Systems. (2025). arXiv:2512.16473 [cs.DC] https://arxiv.org/abs/2512.16473

[15] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-Gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA '24)*. IEEE Press, 1018–1031. doi:10.1109/ISCA59077.2024.00078

[16] Intel. 2024. Intel Resource Director Technology (Intel RDT). https://www.intel.com/content/www/us/en/architecture-and-technology/resource-director-technology.html.

[17] Zhen Jin, Yiquan Chen, Mingxu Liang, Yijing Wang, Guoju Fang, Ao Zhou, Keyao Zhang, Jiexiong Xu, Wenhai Lin, Yiquan Lin, Shushu Zhao, Wenkai Shi, Zhenhua He, Shishun Cai, and Wenzhi Chen. 2025. OS2G: A High-Performance DPU Offloading Architecture for GPU-based Deep Learning with Object Storage. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[18] Elie Kfoury, Samia Choueiri, Ali Mazloum, Ali AlSabeh, Jose Gomez, and Jorge Crichigno. 2024. A Comprehensive Survey on SmartNICs: Architectures, Development Models, Applications, and Research Directions. *IEEE Access* 12 (2024), 107297–107336. doi:10.1109/ACCESS.2024.3437203

[19] Adithya Kumar, Anand Sivasubramaniam, and Timothy Zhu. 2023. SplitRPC: A Control + Data Path Splitting RPC Stack for ML Inference Serving. *Proc. ACM Meas. Anal. Comput. Syst.* 7, 2, Article 30 (2023), 26 pages. doi:10.1145/3589974

[20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*.

[21] Christos P. Lamprakos, Lazaros Papadopoulos, Francky Catthoor, and Dimitrios Soudris. 2022. The Impact of Dynamic Storage Allocation on CPython Execution Time, Memory Footprint and Energy Consumption: An Empirical Study. In *Embedded Computer Systems: Architectures, Modeling, and Simulation*, Alex Orailoglu, Marc Reichenbach, and Matthias Jung (Eds.). Springer International Publishing, Cham, 219–234.

[22] Chris Lattner and Vikram Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. 75–86. doi:10.1109/CGO.2004.1281665

[23] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

[24] Evan Martin. 2012. Ninja: A Small Build System with a Focus on Speed. https://ninja-build.org.

[25] NVIDIA. 2019. NVIDIA TensorRT. https://github.com/NVIDIA/TensorRT.

[26] NVIDIA. 2023. TensorRT-LLM. https://github.com/NVIDIA/TensorRT-LLM.

[27] NVIDIA. 2024. CUDA Graphs. https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cuda-graphs.html.

[28] NVIDIA. 2024. CUDA Graphs — Device Graph Launch. https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cuda-graphs.html#device-graph-launch.

[29] NVIDIA. 2024. CUDA Graphs — Fire and Forget Launch. https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cuda-graphs.html#fire-and-forget-launch.

[30] NVIDIA. 2024. Enabling Dynamic Control Flow in CUDA Graphs with Device Graph Launch. https://developer.nvidia.com/blog/enabling-dynamic-control-flow-in-cuda-graphs-with-device-graph-launch/.

[31] NVIDIA. 2024. NVIDIA-Certified Systems Configuration Guide. https://docs.nvidia.com/certification-programs/latest/nvidia-certified-configuration-guide.html.

[32] NVIDIA. 2026. Nsight Systems. https://developer.nvidia.com/nsight-systems.

[33] OpenChat Team. 2023. OpenChat ShareGPT v3 Dataset. https://huggingface.co/datasets/openchat/openchat_sharegpt_v3.

[34] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. 2019. Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[35] Seo Jin Park, Ramesh Govindan, Kai Shen, David Culler, Fatma Özcan, Geon-Woo Kim, and Hank Levy. 2025. Lovelock: Towards Smart NIC-Hosted Clusters. *SIGENERGY Energy Inform. Rev.* 4, 5 (April 2025), 172–179. doi:10.1145/3727200.3727226

[36] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 8024–8035.

[37] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[38] Yifan Qiao, Shu Anzai, Shan Yu, Haoran Ma, Shuo Yang, Yang Wang, Miryung Kim, Yongji Wu, Yang Zhou, Jiarong Xing, Joseph E. Gonzalez, Ion Stoica, and Harry Xu. 2025. ConServe: Fine-Grained GPU Harvesting for LLM Online and Offline Co-Serving. arXiv:2410.01228 [cs.DC] https://arxiv.org/abs/2410.01228

[39] SGLang Team. 2024. SGLang v0.4: Zero-Overhead Batch Scheduler, FP8 Cache, and More. https://www.lmsys.org/blog/2024-12-04-sglang-v0-4/.

[40] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. (2019). arXiv:1911.02150 [cs.NE] https://arxiv.org/abs/1911.02150

[41] Ying Sheng et al. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

[42] Mohammad Siavashi, Alireza Sanaee, Mohsen Sharifi, and Gianni Antichi. 2025. Phoenix – A Novel Technique for Performance-Aware Orchestration of Thread and Page Table Placement in NUMA Systems. (2025). arXiv:2502.10923 [cs.OS] https://arxiv.org/abs/2502.10923

[43] Vikranth Srivatsa, Dongming Li, Yiying Zhang, and Reyna Abhyankar. 2024. Can Scheduling Overhead Dominate LLM Inference Performance? A Study of CPU Scheduling Overhead on Two Popular LLM Inference Systems. https://mlsys.wuklab.io/posts/scheduling_overhead/.

[44] Ting Sun, Penghan Wang, and Fan Lai. 2025. HyGen: Efficient LLM Serving via Elastic Online-Offline Request Co-location. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[45] Maroun Tork, Lina Maudlej, and Mark Silberstein. 2020. Lynx: A SmartNIC-driven Accelerator-centric Architecture for Network Servers. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM. doi:10.1145/3373376.3378528

[46] Abhishek Vijaya Kumar, Gianni Antichi, and Rachee Singh. 2025. *Aqua: Network-Accelerated Memory Offloading for LLMs in Scale-Up GPU Domains.* Association for Computing Machinery, New York, NY, USA, 48–62. https://doi.org/10.1145/3676641.3715983

[47] vLLM. 2026. *vLLM Documentation — Process Count Summary.* https://docs.vllm.ai/en/v0.16.0/design/arch_overview/#process-count-summary

[48] vLLM. 2026. *vLLM Documentation — V1 Process Architecture.* https://docs.vllm.ai/en/v0.16.0/design/arch_overview/#v1-process-architecture

[49] vLLM Project. 2025. GuideLLM: Evaluate and Enhance Your LLM Deployments for Real-World Inference Needs. https://github.com/vllm-project/guidellm.

[50] vLLM Team. 2025. vLLM V1: A Major Upgrade to vLLM's Core Architecture. https://blog.vllm.ai/2025/01/27/v1-alpha-release.html.

[51] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2026. FastServe: Iteration-Level Preemptive Scheduling for Large Language Models. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[52] Yunming Xiao, Diman Zad Tootaghaj, Aditya Dhakal, Lianjie Cao, Puneet Sharma, and Aleksandar Kuzmanovic. 2024. Conspirator: SmartNIC-Aided Control Plane for Distributed ML Workloads. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 767–784. https://www.usenix.org/conference/atc24/presentation/xiao

[53] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. 2025. Qwen3 Technical Report. (2025). arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[54] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[55] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. https://github.com/deepseek-ai/DeepEP.

[56] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[57] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 193–210. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

Appendix

This appendix provides supplementary evaluation results for Blink. The main paper presents P99 TTFT, P99 TPOT, and goodput under both isolated and colocated conditions; here we extend the analysis to additional percentiles (P99.9, P95, P50, Mean), inter-token latency (ITL), and token-level throughput (prefill and decode tokens/s). All figures show both isolated and CPU-interference conditions across all four models: Llama-3 8B, Phi-4 15B, Qwen-3 32B, and Qwen-3 30B-A3B. We also include summary tables with geometric-mean latency and throughput comparisons over Blink's operating range.

A Experimental Configuration

We briefly recap the experimental setup; full details are in the main paper (§6). All experiments run on a single NVIDIA H100 (96 GB HBM3) server with two Intel Xeon Gold 6336Y CPUs (96 cores), 256 GB DDR5, and a ConnectX-6 200 Gbps NIC. Blink's frontend runs on a BlueField-3 DPU connected via RDMA.

We compare four systems: Blink, TensorRT-LLM v1.1.0 (using the same TensorRT v10.14 engines as Blink), vLLM v0.13.0, and SGLang v0.5.8. The workload uses the ShareGPT v3 dataset with 13 offered-load levels from 1 to 32 req/s. The interference workload colocates *pbzip2* (45 threads) and a *Ninja* LLVM build (45 jobs) on the remaining 90 host cores. Both interference workloads are launched once before the sweep begins and run continuously throughout all 13 load levels. Because these workloads traverse distinct execution phases over their lifetime (Ninja cycles through preprocessing, compilation, assembly, and linking; pbzip2 alternates between I/O-intensive reads and compute-intensive compression blocks), the resource interference they impose varies over time. Consequently, each rate point in the sweep encounters a different interference profile, which can produce non-monotonic fluctuations in baseline interference curves across successive offered loads.

The four models span a range of sizes and architectures: Llama-3 8B (dense, 8B parameters), Phi-4 15B (dense, 14.7B), Qwen-3 32B (dense, 32B), and Qwen-3 30B-A3B (MoE, 30B total / 3B active). In all figures, solid lines denote isolated execution and dashed lines denote execution under CPU interference.

B Pre-Saturation Latency Summary

Table B.1 provides a comprehensive latency comparison across all percentiles. For each model, we report the geometric mean over Blink's operating range ($\lambda \leq 12$ for Llama-3 8B, $\lambda \leq 7$ for Phi-4 15B, $\lambda \leq 2$ for Qwen-3 32B, $\lambda \leq 4$ for Qwen-3 30B-A3B) under isolated execution. The geometric mean is used (consistent with the main paper) because it is less sensitive to a single high-load outlier and is appropriate for ratio-scale latency data.

P50 TTFT. Blink achieves the lowest median TTFT on all models. On Llama-3 8B, baselines incur 1.73× (TensorRT-LLM, 72.3 ms) to 5.75× (SGLang, 240.3 ms) higher median TTFT than Blink (41.8 ms). The Qwen-3 30B-A3B result is notable: TensorRT-LLM's 1 132 ms median TTFT is 5.5× higher than Blink's 207 ms, reflecting the high cost of CPU-mediated expert routing at even the median.

Mean latency. Mean TTFT and TPOT track between P50 and P95, as expected for heavy-tailed queuing distributions. On Phi-4 15B, Blink achieves a mean TTFT of 258.8 ms compared to 355.7 ms for TensorRT-LLM (1.37×) and 846.9 ms for SGLang (3.27×). Mean TPOT differences are smaller but consistent: Blink's per-token decode overhead is the lowest or near-lowest on every model.

Table B.1. Geometric-mean latency (ms) over Blink's operating range under isolated execution. Best values per model are boldfaced. Operating ranges: $\lambda \leq 12$ (Llama-3 8B), ≤ 7 (Phi-4 15B), ≤ 2 (Qwen-3 32B), ≤ 4 (Qwen-3 30B-A3B).

Model	System	P50 TTFT	Mean TTFT	P50 TPOT	Mean TPOT
Llama-3 8B	Blink	**41.8**	**116.9**	**7.5**	**8.2**
	TRT-LLM	72.3	170.1	8.6	9.6
	vLLM	111.1	276.1	11.0	12.4
	SGLang	240.3	507.1	12.9	14.5
Phi-4 15B	Blink	**105.8**	**258.8**	**13.4**	**14.1**
	TRT-LLM	153.1	355.7	15.2	16.3
	vLLM	229.5	455.8	16.8	18.1
	SGLang	540.9	846.9	20.6	22.4
Qwen-3 32B	Blink	786.2	2501	**29.7**	**35.9**
	TRT-LLM	**531.7**	**2344**	31.0	37.2
	vLLM	756.0	3009	32.7	40.8
	SGLang	788.5	2896	34.2	40.9
Qwen-3 30B-A3B	Blink	**207.1**	**426.1**	**11.9**	**13.8**
	TRT-LLM	1132	1405	21.3	24.3
	vLLM	1507	2620	25.6	30.9
	SGLang	1778	3441	28.4	35.2

Qwen-3 32B: the GPU-bound regime. On the largest dense model, differences between systems are smallest across all percentiles. This is consistent with the main paper's finding that once the workload becomes predominantly GPU-bound, there is less host-side latency to remove. Nevertheless, Blink maintains parity or a slight edge, and the advantage re-emerges at P99.9 (§D.1 below).

Table B.2 shows token-level throughput at each model's saturation point. Decode throughput is the most scheduling-sensitive metric: each autoregressive step requires a scheduler iteration, so CPU-mediated overhead compounds linearly with output length. Blink achieves 3 880 tok/s decode throughput on Llama-3 8B at saturation (10% above TensorRT-LLM) and 1 437 tok/s on Qwen-3 30B-A3B (36% above TensorRT-LLM).

C Serviceable Load Summary

Fig. C.1 summarizes the maximum serviceable load for each system-model pair under isolated (a) and interference (b) conditions. The serviceable load is defined as the highest offered rate at which the system retains at least 95% of the ideal throughput (*i.e.,* goodput $\geq 0.95 \times$ offered rate). This metric captures the practical deployment capacity: the maximum request rate at which a system can operate without significant queuing or request dropping.

Blink achieves the highest serviceable load on every model under both isolated and interference conditions. On the MoE model (Qwen-3 30B-A3B), the gap is particularly large: Blink sustains nearly twice the serviceable load of the nearest baseline under isolation, and the advantage widens further under interference where CPU-mediated systems lose scheduling capacity to the competing workloads. On Llama-3 8B, the serviceable load under interference drops by 60–65% for baselines while Blink retains its full isolated capacity.

Table B.2. Token-level throughput (tok/s) at BLINK's saturation point under isolated execution. Best values per model are boldfaced.

Model	System	Decode tok/s	Prefill tok/s
Llama-3 8B	BLINK	**3880**	**595**
	TRT-LLM	3535	582
	vLLM	2930	564
	SGLang	2638	553
Phi-4 15B	BLINK	**2177**	**465**
	TRT-LLM	2044	459
	vLLM	1906	451
	SGLang	1690	444
Qwen-3 32B	BLINK	**537**	**236**
	TRT-LLM	520	235
	vLLM	487	229
	SGLang	482	229
Qwen-3 30B-A3B	BLINK	**1437**	**374**
	TRT-LLM	1053	318
	vLLM	841	262
	SGLang	730	257

D Extended Latency Metrics

The main paper reports P99 tail latency for TTFT and TPOT (Fig. 6). Here we present four additional percentile breakdowns: P99.9, P95, P50 (median), and Mean. Each figure covers all four models under both isolated and interference conditions. P99.9 and P95 figures show TTFT and TPOT; P50 and Mean figures also include ITL.

These percentiles serve complementary purposes. P99.9 exposes outlier behavior at the deepest practical tail; P95 captures the experience of the vast majority of requests without the noise sensitivity of P99.9; P50 (median) reflects the typical-case latency; and the mean provides an aggregate view useful for capacity planning. ITL (inter-token latency) is relevant for streaming applications where users perceive token-by-token delivery smoothness.

D.1 P99.9 Tail Latency

Fig. D.1 extends the P99.9 analysis from the main paper to all four models.

Isolation. At this deep tail, BLINK consistently maintains lower latency across the full load range. The separation is most pronounced on Qwen-3 30B-A3B, where MoE expert-routing overhead in CPU-mediated systems amplifies tail events. On Llama-3 8B, baselines incur 1.3–2.6× higher P99.9 TTFT than BLINK within the operating range, consistent with the P99 ratios reported in the main paper.

Interference. Under CPU contention, baseline P99.9 latencies diverge sharply from their isolated counterparts (dashed vs. solid curves), while BLINK's curves remain nearly overlapping. This confirms that the interference immunity documented at P99 in the main paper extends to the deepest practical tail. The effect is especially visible in the TPOT row (e–h), where per-iteration scheduling delays compound across hundreds of decode steps.

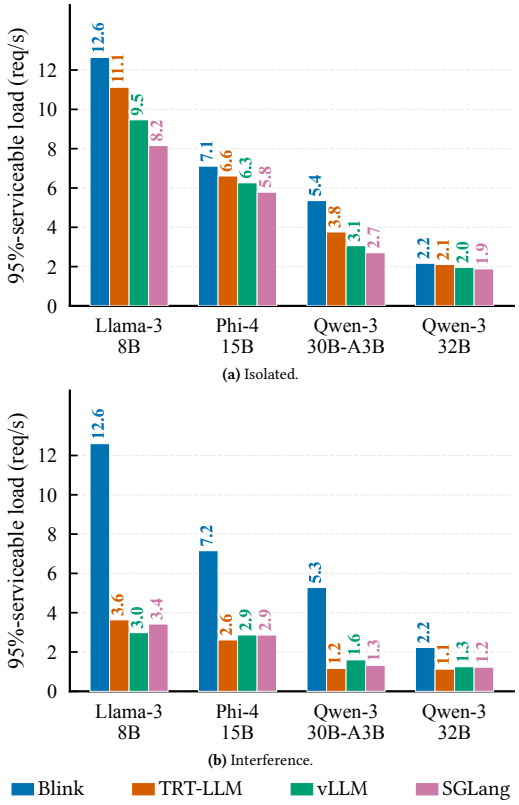


Figure C.1. Maximum serviceable load (95% throughput retention threshold) per system and model. Higher is better. BLINK achieves the highest serviceable load on every model; on Qwen-3 30B-A3B the gap is largest under isolation, and under interference BLINK retains its full isolated capacity while baselines lose 60–65% on Llama-3 8B.

D.2 P95 Tail Latency

Fig. D.2 reports P95 latency. At this percentile, the relative ordering among systems is consistent with P99 (main paper), but absolute values are lower and the curves are smoother due to reduced sensitivity to individual outlier requests.

Isolation. BLINK retains its latency advantage across all models. The P95 TTFT gaps are proportionally similar to P99: on Llama-3 8B, baselines incur 1.3–2.4× higher P95 TTFT than BLINK within the operating range. This confirms that the benefit is structural (reduced scheduling overhead) rather than an artifact of rare tail events.

Interference. The separation between isolated and interference curves is visible at P95 for all CPU-mediated baselines, while BLINK's P95 curves remain stable. Under interference, baseline P95 TPOT rises steeply past saturation, directly impacting per-token decode latency for interactive applications.

D.3 P50 (Median) Latency

Fig. D.3 shows median latency. Median values are important for capacity planning because they reflect the experience of a typical request rather than worst-case outliers.

Isolation. BLINK achieves the lowest or near-lowest median across all metrics and models. On Qwen-3 32B, where the workload is heavily GPU-bound, median differences between systems are small

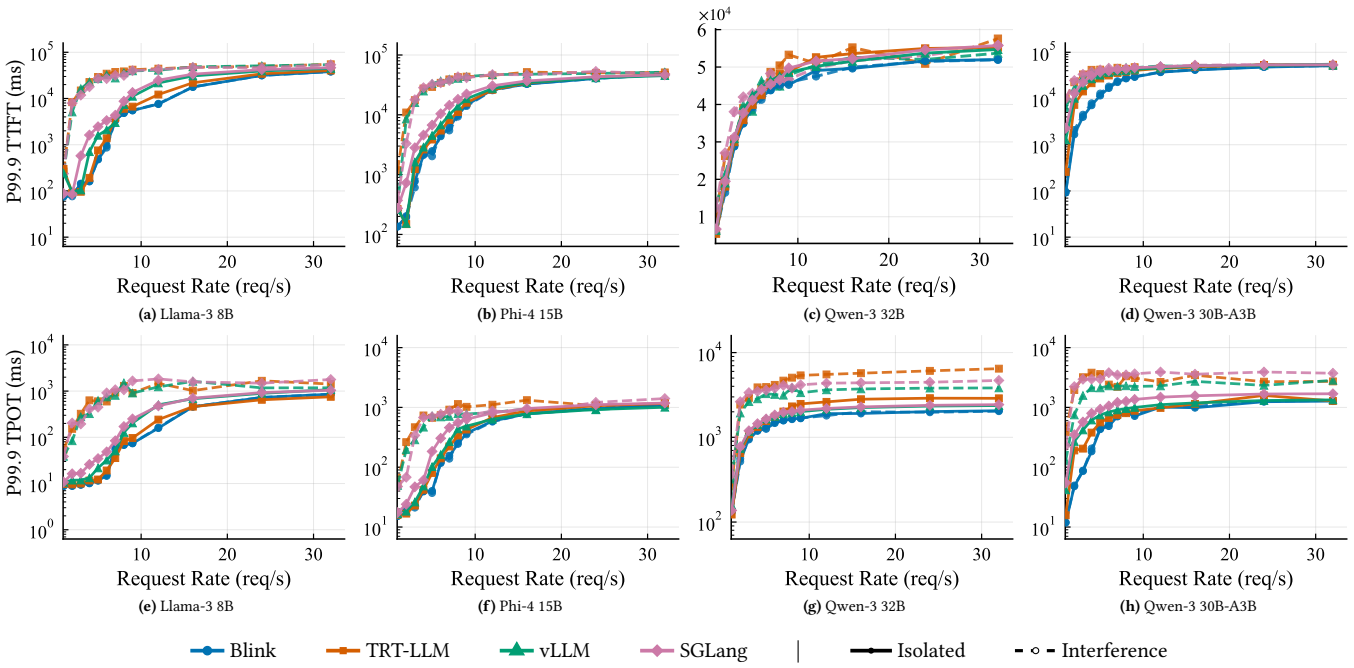


Figure D.1. P99.9 tail latency across four models. Top row (a–d): TTFT; bottom row (e–h): TPOT. Solid lines show isolated execution; dashed lines show CPU interference. Blink sustains lower P99.9 latency across the full load sweep on all four models, with the largest margin on the MoE model (Qwen-3 30B-A3B). Baseline dashed curves separate sharply from their isolated counterparts, while Blink's remain nearly overlapping.

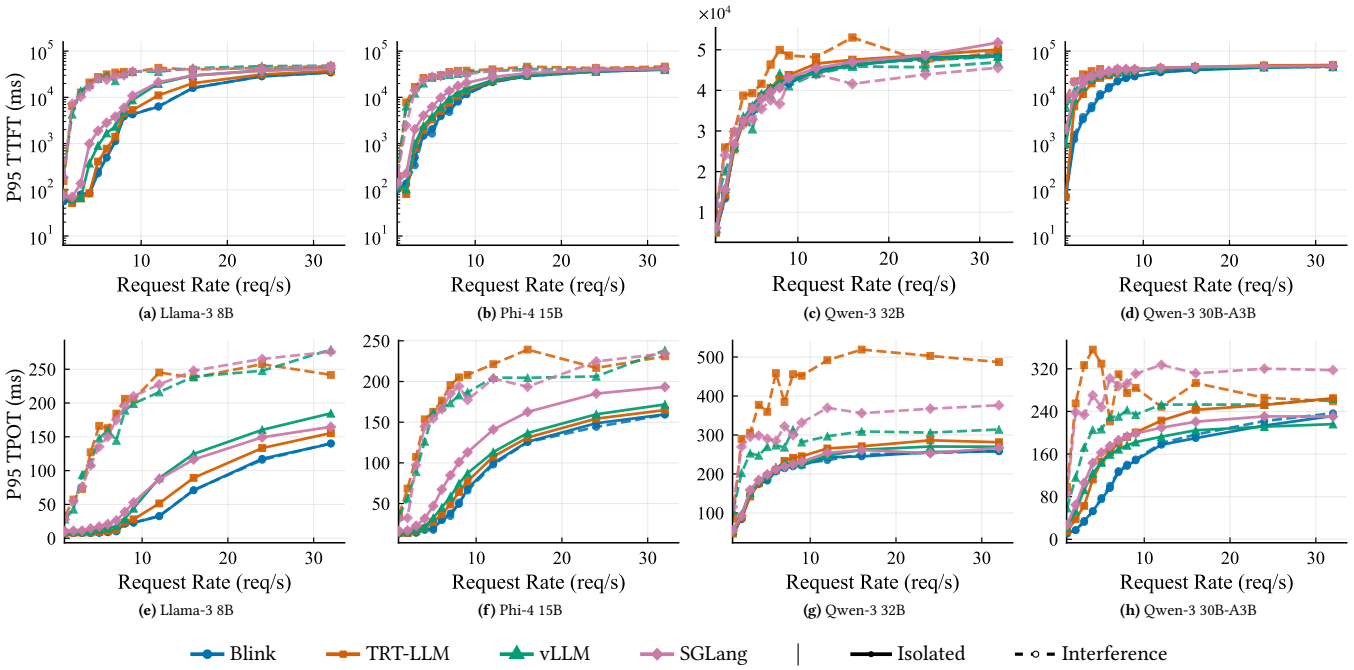


Figure D.2. P95 latency across four models. Top row (a–d): TTFT; bottom row (e–h): TPOT. Solid lines show isolated execution; dashed lines show CPU interference. The relative system ordering is consistent with the P99 results in the main paper, confirming that Blink's advantage is structural rather than an artifact of rare outliers.

(Table B.1), consistent with the P99 observations in the main paper. On Qwen-3 30B-A3B the picture is different: TensorRT-LLM's median TTFT (1 132 ms) is 5.5× higher than Blink's (207 ms), confirming that expert-routing overhead is not confined to the tail.

Interference. At the median, CPU interference shifts baseline curves upward significantly. Baselines that were within 2× of Blink

in isolation can exceed 10× under interference. Blink's median remains essentially unchanged, demonstrating that the typical request is also protected, not only tail percentiles.

D.4 Mean Latency

Fig. D.4 reports mean latency. Because the mean is sensitive to heavy-tailed distributions (common in queuing systems under load), it tends to track between the median and P95. The mean is included here for readers who use it in SLO definitions or cost models.

Isolation. System ordering is consistent with both P50 and P99. On Qwen-3 30B-A3B, baselines incur 3.3× (TensorRT-LLM) to 8.1× (SGLang) higher mean TTFT than Blink (426 ms) (Table B.1). The mean captures not only the typical case but also the impact of occasional long-tail requests, making it a useful single-number summary for system comparison.

Interference. Under interference (dashed lines), mean latency shows the same pattern documented at P99 in the main paper: baseline means inflate by 3–18× while Blink's mean stays within 1.0–1.15× of isolation. Blink's steady-state path does not traverse the contended host CPU, so even average-case latency is structurally decoupled from host load.

E Extended Throughput Metrics

The main paper reports goodput (completed requests per second). Here we provide a complementary view: token-level throughput, broken down into prefill (prompt tokens processed per second) and decode (output tokens generated per second). These metrics are relevant for deployments that bill or provision by token count rather than by request.

Fig. E.1 shows both metrics across all four models. Prefill throughput reflects how quickly the system can ingest prompts and is primarily compute-bound. Decode throughput captures autoregressive token generation rate and is sensitive to scheduling overhead, since each decode step requires a scheduler iteration.

Decode throughput. Blink's GPU-resident scheduler eliminates per-iteration CPU round-trips, which translates into higher sustained decode throughput. The advantage is most visible on Qwen-3 30B-A3B (Table B.2), where Blink sustains 1 437 tok/s at saturation, 36% above TensorRT-LLM (1 053 tok/s) and 97% above SGLang (730 tok/s).

Prefill throughput. Prefill throughput differences are smaller because the prefill phase is a single large matrix multiplication that is less affected by scheduling overhead. Nevertheless, Blink maintains a consistent edge (2–18% above the nearest baseline) because it avoids the CPU-side latency of marshaling prefill requests.

Interference. Under CPU contention, baseline decode throughput drops significantly (dashed curves in the bottom row), while Blink's throughput plateau is preserved. This mirrors the goodput findings in the main paper and confirms that the throughput collapse under interference is not specific to the request-level metric but extends to the token level.

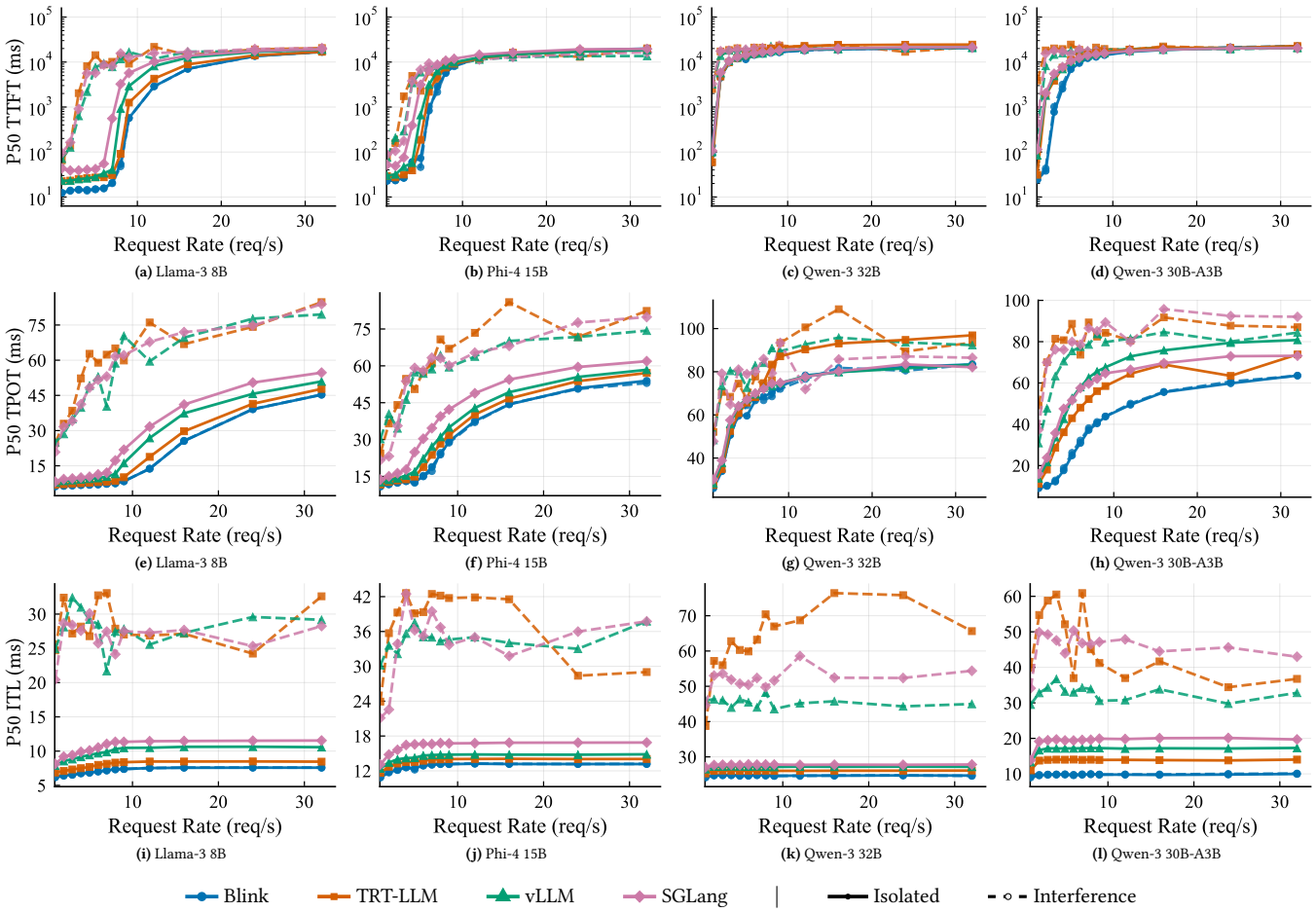


Figure D.3. Median (P50) latency across four models. Top row (a–d): TTFT; middle row (e–h): TPOT; bottom row (i–l): ITL. Solid lines show isolated execution; dashed lines show CPU interference. BLINK achieves the lowest or near-lowest median across all metrics, with differences compressing on the GPU-bound Qwen-3 32B. Under interference, baseline medians inflate substantially while BLINK's remain stable.

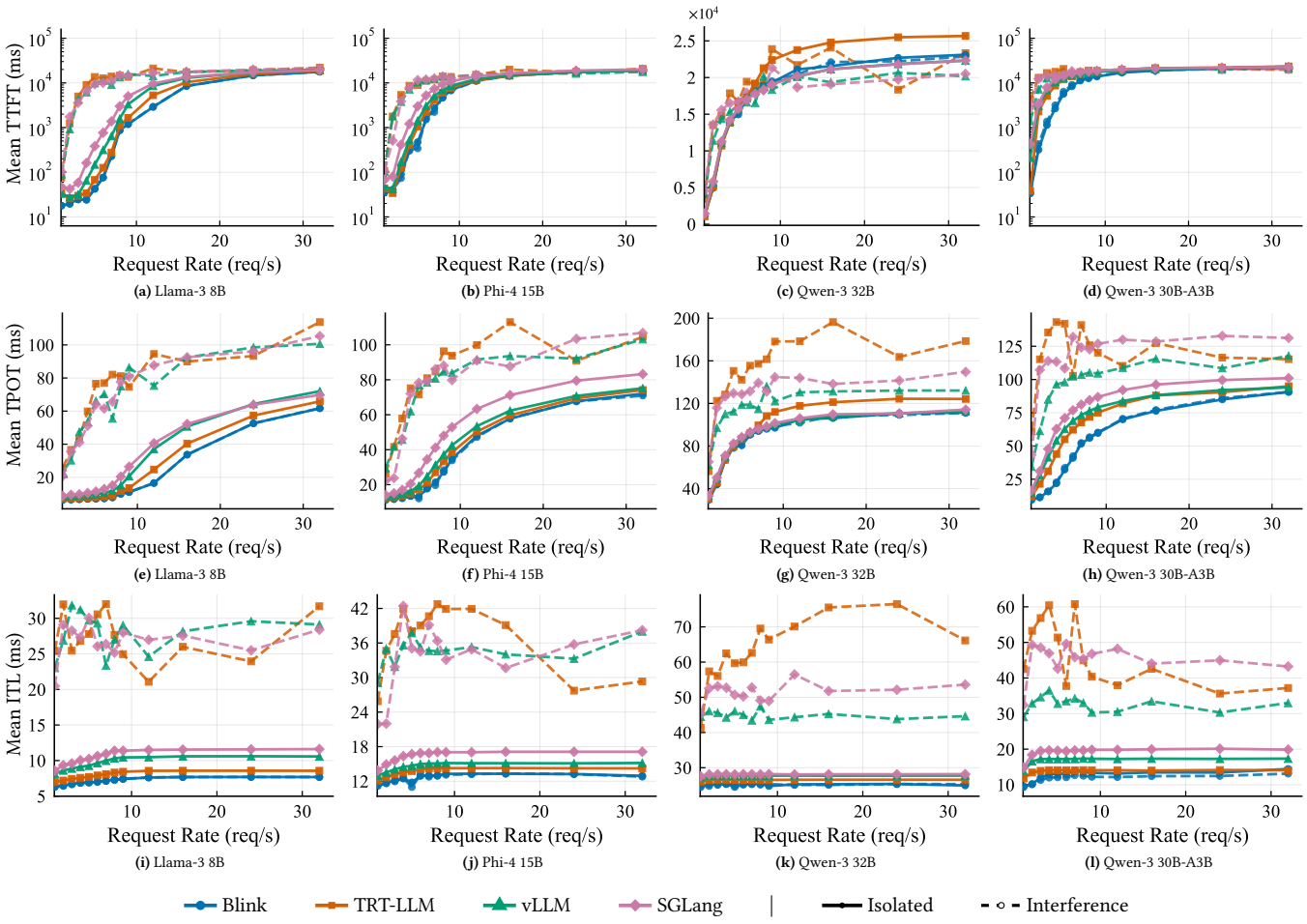


Figure D.4. Mean latency across four models. Top row (a–d): TTFT; middle row (e–h): TPOT; bottom row (i–l): ITL. Solid lines show isolated execution; dashed lines show CPU interference. The mean tracks between P50 and P95 due to heavy-tailed queuing distributions; system ordering remains consistent with tail percentile results. Baseline means inflate by 3–18× under interference while BLINK's stays within 1.0–1.15×.

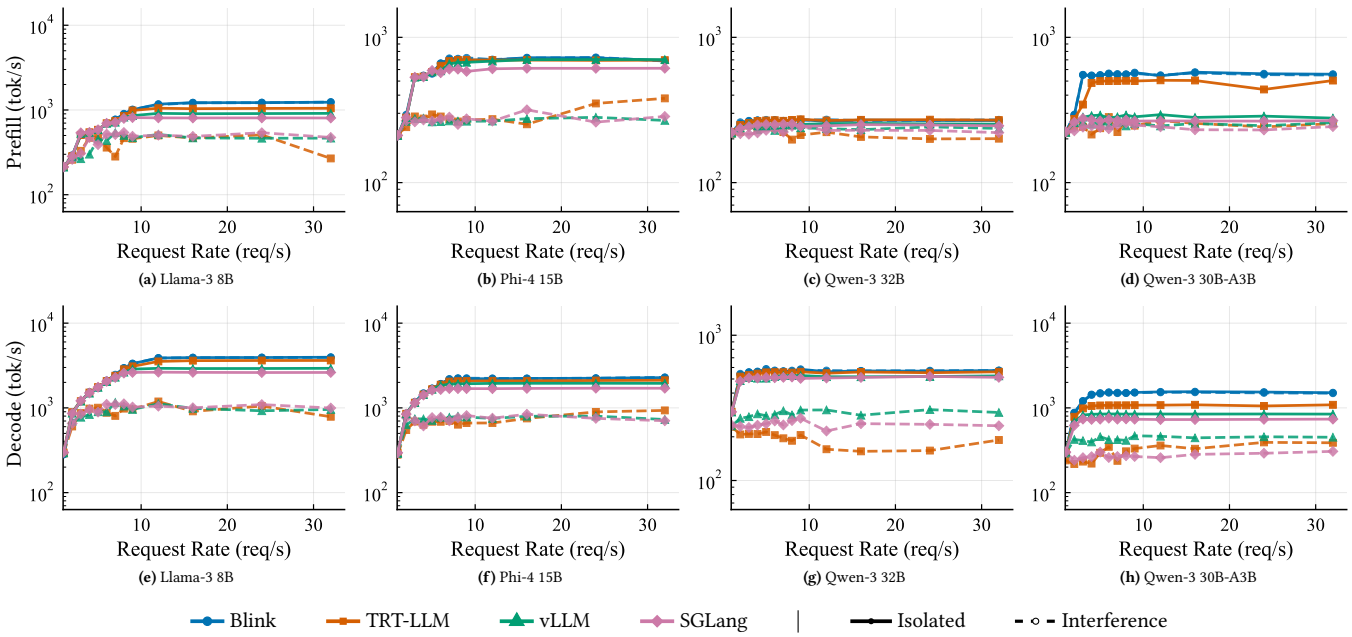


Figure E.1. Token-level throughput across four models. Top row (a–d): prefill throughput (prompt tokens/s); bottom row (e–h): decode throughput (output tokens/s). Solid lines show isolated execution; dashed lines show CPU interference. Blink's GPU-resident scheduling eliminates per-iteration CPU round-trips, yielding higher sustained decode throughput. Under interference, baseline decode throughput collapses while Blink's plateau is preserved.